

HNSW with Accuracy Guarantees Using Graph Spanners

Minghao Li
University of Toronto
Toronto, Canada
minghao.li@mail.utoronto.ca

Raghav Mittal
The University of Texas At Arlington
Arlington, Texas, USA
rxm0006@mavs.uta.edu

Sanjivni Rana
The University of Texas At Arlington
Arlington, Texas, USA
srx0277@mavs.uta.edu

Suraj Shetiya
IIT Bombay
Mumbai, India
surajs@cse.iitb.ac.in

Gautam Das
The University of Texas at Arlington
Arlington, Texas, USA
gdas@uta.edu

Nick Koudas
University of Toronto
Toronto, Canada
koudas@cs.toronto.edu

ABSTRACT

Hierarchical Navigable Small World (HNSW) graphs serve as the industry standard due to their logarithmic complexity and strong empirical performance. However, HNSW relies on greedy graph traversal, a heuristic that provides no theoretical guarantees of correctness. In this paper, we propose a novel "Certify-then-Rectify" framework that bridges the gap between the speed of heuristic search and the rigor of exact retrieval. Rather than discarding HNSW, our approach first employs a distribution-free statistical certifier to dynamically evaluate the quality of a standard HNSW search with minimal overhead. If the certification indicates that the retrieved neighbors are of low quality, the framework safely escalates to a rigorous exact recovery algorithm. To make this exact recovery computationally feasible, we reinterpret the HNSW graph as a geometric spanner and utilize Extreme Value Theory to stochastically estimate its maximum empirical stretch factor. This allows us to mathematically bound the maximum distance of true nearest neighbors. Furthermore, we successfully extend these theoretical guarantees to filtered search scenarios. Extensive evaluations on benchmark datasets demonstrate that our tiered framework delivers the average-case speed of HNSW while ensuring the worst-case correctness of exact search.

ACM Reference Format:

Minghao Li, Raghav Mittal, Sanjivni Rana, Suraj Shetiya, Gautam Das, and Nick Koudas. 2026. HNSW with Accuracy Guarantees Using Graph Spanners. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 22 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Nearest Neighbor Search (NNS) is the computational engine behind modern AI, driving workloads ranging from Large Language Model (LLM) retrieval to molecular simulations. As datasets scale

into the billions of vectors, exact linear scanning becomes computationally prohibitive, necessitating Approximate Nearest Neighbor (ANN) algorithms. Among these, Hierarchical Navigable Small World (HNSW) graphs [37, 40] have become the industry standard due to their logarithmic complexity and generally strong empirical performance.

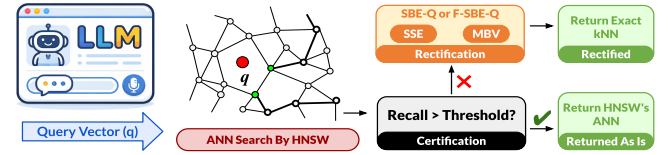


Figure 1: Certify-then-Rectify (CTR) search framework

However, HNSW relies on a greedy traversal of a proximity graph—a heuristic that offers no theoretical guarantee of recall [2, 12, 45, 55]. While HNSW performs well empirically, it is highly sensitive to hyperparameter selection (e.g., beam width, construction depth) [20, 53]. In high-dimensional spaces, greedy search frequently gets trapped in local minima, failing to retrieve the true nearest neighbor due to topology disconnects or the "curse of dimensionality." While a 95% recall rate is acceptable for e-commerce, it is fundamentally inadequate for logic-driven and scientific domains where a single missing entry can invalidate a result.

The demand for **Exact kNN** is growing rapidly in complex, high-stakes applications: In training perception models, identifying "hard negatives"—images that look safe but are actually dangerous—is critical [14, 26]. If an ANN algorithm fails to retrieve these specific edge cases during data mining, the training set remains noisy, compromising safety. Exact retrieval is required to rigorously clean training distributions. In systems combining neural retrieval with symbolic logic or LLMs, vector search acts as a predicate selector. Missing a specific "bridge" entity (e.g., a specific legal precedent or protein structure) leads to "hallucinations" or false logical implications because the reasoning engine operates on incomplete premises [34, 48].

Existing attempts to mitigate these failures by simply increasing or predicting [12, 20] HNSW hyperparameters yield diminishing returns, incurring latency penalties without ever providing a guarantee of correctness. Moreover, they impose retraining overheads under distribution shifts.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

We propose a novel framework that bridges the gap between the speed of heuristic search and the rigor of exact retrieval. We acknowledge that while our proposed search method, **MBV** (Metric Bound Verification) (utilizing our proposed Stretch Bounded Expansion from Query), accurately provides a formal probabilistic guarantee, it is computationally slower than heuristic HNSW. Therefore, rather than discarding HNSW, we wrap it in a **Certify-then-Rectify (CTR)** framework.

Our approach allows users to first **certify** the quality of a standard HNSW search result with negligible overhead. If—and only if—the certification indicates that the retrieved neighbors are of low quality, we trigger the computationally heavier MBV algorithm to identify the exact kNN, with high tunable probability¹. This tiered strategy provides the best of both worlds: the average-case speed of HNSW and the worst-case correctness of exact search (see Figure 1).

Our contributions are structured as follows (see Figure 1):

1. Stochastic Stretch Estimation (SSE): To enable exact search, we reinterpret the HNSW graph as a geometric spanner. We introduce SSE, a sampling algorithm that accurately approximates the graph's stretch factor t . This factor allows us to mathematically bound the maximum distance the true nearest neighbor could reside from our approximate candidates, establishing a dynamic search radius.

2. Stretch-Bounded Expansion (SBE-Q): We present SBE-Q a technique to recover the exact neighbors. Unlike greedy heuristic descent, SBE-Q performs a principled expansion.

- **SBE from the Query Point (SBE-Q):** We confine the search to a radius defined by the distance to the k -th approximate neighbor (d_k) multiplied by the stretch factor t (i.e., $\text{radius} = t \times d_k$). This guarantees the discovery of the true nearest neighbors.
- **Reservoir Sampling:** To ensure SBE-Q remains theoretically sound, we use reservoir sampling to virtually "insert" the query point q into the stretch estimation sample without altering the graph's global stretch properties.

We also explore a baseline variant, SBE-NN (Expansion from Nearest Neighbor), but find SBE-Q significantly more efficient due to its tighter bounding radius.

3. Metric Bound Verification (MBV): Finally, we refine the candidate set obtained by SBE-Q using triangular inequality bounds derived from the stretch factor. This step recycles distance computations from the initial search to strictly filter points, ensuring the final output is the exact k -NN set. Our approach will work for any distance metric that directly or indirectly satisfies triangle inequality.

4. Utility in Filtered Search: We further demonstrate that SBE-Q naturally adapts to filtered search (e.g., "nearest vector where property P is true"), addressing scenarios where filtering fractures the small-world topology. Our techniques can be utilized within extensions like ACORN [40] to prevent traversals from getting

¹In the sequel we use the term exact search when deploying MBV, with the understanding that the probability to identify the true kNN can be made arbitrarily high as per Section 6.4 evaluated further empirically in Section 7.

stranded in disconnected subgraphs. Theoretical details are in Section 8.

5. Certification of HNSW Results: We introduce a post-hoc, distribution-free statistical certification framework for vanilla HNSW. By leveraging Conformal Risk Control (CRC) [3, 4] and Learn-then-Test (LTT) [5] procedures, we construct a rigorous certifier that evaluates a distance-margin score derived purely from HNSW's internal search state. This approach establishes finite-sample statistical guarantees on expected recall shortfall and provides per-query probabilistic bounds. Ultimately, this certifier acts as a dynamic gatekeeper with minimal computational overhead, determining whether to confidently accept the fast heuristic results or to safely escalate to exact recovery via MBV. The entire approach is realized as a lightweight wrapper around existing HNSW implementations, facilitating ease of use.

2 PRELIMINARIES

In this section, we formalize the problem of Nearest Neighbor Search (NNS) and the structure of Hierarchical Navigable Small World (HNSW) graphs. We establish the notation for the search procedure and its computational artifacts, and we introduce the concept of graph spanners and stretch factors, which form the theoretical basis of our proposed method.

Let $\mathcal{X}$ be a dataset of n points in a d -dimensional vector space equipped with a distance metric $\text{dist}(\cdot, \cdot)$ (typically Euclidean distance). Given a query q and an integer k , the Exact k -Nearest Neighbor (k -NN) problem is to find the set $\mathcal{N}_k^*(q) \subseteq \mathcal{X}$ such that $|\mathcal{N}_k^*(q)| = k$ and $\forall x \in \mathcal{N}_k^*(q), \forall y \in \mathcal{X} \setminus \mathcal{N}_k^*(q), \text{dist}(q, x) \leq \text{dist}(q, y)$.

The HNSW index is a hierarchical graph structure approximating the proximity of points in $\mathcal{X}$. It consists of a series of layers $L_0, L_1, \dots, L_{l_{\max}}$, where layer L_0 contains all data points, and each subsequent layer L_i contains a subset of the points from layer L_{i-1} , forming a hierarchy. Formally, an HNSW structure is a tuple of graphs $\mathcal{H} = (G_0, G_1, \dots, G_{l_{\max}})$. Each graph $G_i = (V_i, E_i)$ is defined such that $V_i \subseteq \mathcal{X}$. The edges E_i connect points within the same layer based on proximity heuristics. The hierarchy allows for logarithmic search complexity by initiating the search at the top layer (coarse granularity) and progressively refining the search region as traversal moves down to layer L_0 (fine granularity).

Construction proceeds iteratively by inserting points $x \in \mathcal{X}$. For each x , a maximum layer $l(x) \leq l_{\max}$ is selected randomly based on an exponential distribution. The point is inserted into all graphs $G_0, G_1, \dots, G_{l(x)}$. Connections in each layer are established using a heuristic selection strategy that balances proximity with spatial diversity (connectivity) to ensure the "small world" property, typically bounded by a maximum degree parameter M .

The search for the k -NN of a query q in HNSW is a greedy traversal. Starting from an entry point e_{top} in the top layer $L_{l_{\max}}$, the algorithm greedily moves to the neighbor minimizing $\text{dist}(q, \cdot)$ until a local minimum is reached. This process repeats down to layer L_0 . In layer L_0 , a beam search is conducted with a beam width parameter ef . Let this process be denoted as $\text{Search}(q, k, ef)$. The output of this heuristic search is an approximate set $\mathcal{N}_k(q) \subseteq \mathcal{X}$, where often $|\mathcal{N}_k(q)| < k$ and $\mathcal{N}_k(q) \cap \mathcal{N}_k^*(q)$ may be empty in worst-case scenarios. Crucially, our approach utilizes the artifacts

generated during this traversal. We define the Search Trace $\mathcal{T}(q)$ as the set of all points for which the distance to q was computed during the search:

$$\mathcal{T}(q) = \{x \in \mathcal{X} \mid \text{dist}(q, x) \text{ was computed during Search}(q, k, ef)\}.$$

This set includes the visited nodes and their immediate neighbors evaluated during the beam search. To bound the error of the approximate search, we analyze the bottom-layer graph G_0 as a geometric spanner.

Definition 2.1. A weighted graph $G = (V, E)$, where vertices are points in a metric space $(\mathcal{X}, \text{dist})$, is a t -spanner if, for every pair of vertices $u, v \in V$, the shortest path distance in the graph, denoted $d_G(u, v)$, satisfies:

$$\text{dist}(u, v) \leq d_G(u, v) \leq t \cdot \text{dist}(u, v).$$

The value $t \geq 1$ is called the stretch factor (or dilation) of the spanner.

In the context of NNS, the stretch factor limits how much the graph topology distorts the true metric space. If G_0 were a perfect 1-spanner (a complete graph with edge weights equal to metric distances), greedy search would be exact. The sparsification required for efficiency introduces distortion. While HNSW construction heuristics do not theoretically guarantee a constant stretch factor t for arbitrary worst-case inputs (i.e., it is not a t -spanner by design in the traditional computational geometry sense), any realized finite HNSW graph instance G_0 possesses an intrinsic, maximum empirical stretch t_{emp} defined as:

$$t_{\text{emp}} = \max_{u, v \in \mathcal{X}} \frac{d_{G_0}(u, v)}{\text{dist}(u, v)}.$$

This property is pivotal: if we know (or estimate) t_{emp} , we can relate the graph distance—which we can explore efficiently via Breadth-First Search (BFS) or Dijkstra—to the unknown metric distance $\text{dist}(q, x)$, thereby establishing a stopping condition for search expansion that guarantees the inclusion of $\mathcal{N}_k^*(q)$.

3 STOCHASTIC STRETCH ESTIMATION

For graphs of realistic sizes, computing the stretch factor exactly is not computationally feasible, as it would involve computations quadratic to the number of nodes in the graph. We, in turn, present sampling based algorithms to estimate the stretch factor of an HNSW graph G .

Let S be a random variable representing the stretch factor of a uniformly random pair (u, v) from $V \times V \setminus \{(v, v) : v \in V\}$. Rather than estimating arbitrary percentiles, our goal is to estimate the absolute maximum empirical stretch factor via a finite sample.

Definition 3.1 (Sample Block Maxima). Let the total sampled pairs be divided into m blocks of size n . Let $M_{n,i} = \max\{S_{i,1}, \dots, S_{i,n}\}$ be the maximum stretch factor observed in the i -th block.

THEOREM 3.2 (EVT MAXIMUM STRETCH BOUND). *According to the Fisher-Tippett-Gnedenko theorem [24], the distribution of the maximum of a large sequence of independent, identically distributed random variables converges to a specific family of distributions. For the right tail of the stretch factor distribution, the cumulative distribution*

function (CDF) of the block maxima converges to the Generalized Extreme Value (GEV) distribution:

$$H(s; \mu, \sigma, \xi) = \exp \left(- \left[1 + \xi \left(\frac{s - \mu}{\sigma} \right) \right]^{-1/\xi} \right)$$

where μ is the location parameter, $\sigma > 0$ is the scale parameter, and ξ is the shape parameter.

By modeling the “right tail” (the largest observed stretch factors) of our sample, we can accurately estimate the unobserved probability density of extreme stretch distortions. For a given confidence level $\beta \in (0, 1)$, the estimated maximum stretch factor t is defined as the return level derived from the inverse CDF of the fitted GEV distribution:

$$t = \mu - \frac{\sigma}{\xi} \left[1 - (-\ln \beta)^{-\xi} \right]$$

Thus, with confidence β , the true empirical stretch factor of the graph is bounded by t .

3.1 Practical Application of the EVT Bound

In practice, applying Theorem 3.2 requires a systematic sampling and fitting procedure prior to initiating the search. First, we generate a large pool of independent stretch factor samples by computing the exact metric distance and the shortest-path graph distance for random pairs of vertices in G . This pool is then partitioned into m distinct blocks of size n . We extract the maximum stretch value from each block, resulting in a specialized dataset composed entirely of block maxima. Next, we apply Maximum Likelihood Estimation (MLE) to this dataset to find the optimal parameters for the Generalized Extreme Value distribution: the location μ , scale σ , and shape ξ . Once the MLE converges and the parameters are established, we substitute them—along with our target confidence level β (e.g., 0.99)—into the GEV return level equation. The resulting value, t , acts as the rigorously defined operating stretch factor for our subsequent search expansions, ensuring the search radius dynamically adapts to the graph’s most extreme topological distortions.

4 GRAPH BASED PRUNING TECHNIQUES

4.1 Stretch Bounded Expansion from Nearest Neighbor (SBE-NN)

We consider $(\mathcal{X}, \text{dist})$ to be a metric space, where $G_0 = (\mathcal{X}, E)$ corresponds to the bottom-layer HNSW graph. Notably, this can be viewed as a *weighted* graph where each edge $(u, v) \in E$ has weight $\text{dist}(u, v)$. We denote the shortest-path distance in G_0 (sum of edge weights) as $d_{G_0}(u, v)$.

Assuming G_0 to be a t -spanner (globally or locally in the vicinity of q) in the sense that for all relevant pairs u, v :

$$\text{dist}(u, v) \leq d_{G_0}(u, v) \leq t \cdot \text{dist}(u, v)$$

To analyze the search expansion, we must distinguish between the approximate candidates returned by the HNSW index and the unknown ground truth.

For a given query q , let the ordered set of k candidates returned by the initial HNSW search be $\{x_{(1)}, x_{(2)}, \dots, x_{(k)}\}$. We define the distances to the nearest and k -th candidates as:

$$\hat{d}_1 = \text{dist}(q, x_{(1)}) \quad \text{and} \quad \hat{d}_k = \text{dist}(q, x_{(k)})$$

Let $N_k^*(q)$ denote the set of the true top- k nearest neighbors. Let d_k^* denote the distance to the true k -th nearest neighbor:

$$d_k^* = \max_{x \in N_k^*(q)} \text{dist}(q, x)$$

Since the HNSW candidates $\{x_{(1)}, \dots, x_{(k)}\}$ form a valid subset of $\mathcal{X}$ of size k , and the true set $N_k^*(q)$ minimizes the sum of distances, it necessarily holds that the true k -th distance is bounded by the approximate k -th distance:

$$d_k^* \leq \hat{d}_k \quad (1)$$

This observation allows us to construct a search radius based on the *observed* HNSW results that guarantees the inclusion of the *unknown* true neighbors as per Figure 2.

THEOREM 4.1 (STRETCH-BOUNDED CONTAINMENT OF TRUE TOP-K). *Given a query q , let $x_{(1)}$ be the nearest candidate and $\hat{d}_k$ be the distance to the k -th candidate returned by HNSW. Every point in the true top- k set $N_k^*(q)$ lies inside the graph ball of radius $t(\hat{d}_1 + \hat{d}_k)$ centered at $x_{(1)}$.*

Formally, for every $x^* \in N_k^*(q)$:

$$d_{G_0}(x_{(1)}, x^*) \leq t(\hat{d}_1 + \hat{d}_k)$$

Consequently:

$$N_k^*(q) \subseteq \{x \in \mathcal{X} : d_{G_0}(x_{(1)}, x) \leq t(\hat{d}_1 + \hat{d}_k)\}$$

PROOF. Let x^* be any point in the true set $N_k^*(q)$. By definition of the true k -th distance and Equation 1:

$$\text{dist}(q, x^*) \leq d_k^* \leq \hat{d}_k \quad (2)$$

Using the triangle inequality in the metric space:

$$\text{dist}(x_{(1)}, x^*) \leq \text{dist}(x_{(1)}, q) + \text{dist}(q, x^*) \quad (3)$$

Substituting known variables $\text{dist}(x_{(1)}, q) = \hat{d}_1$ and the bound from Equation 2:

$$\text{dist}(x_{(1)}, x^*) \leq \hat{d}_1 + \hat{d}_k$$

This represents the maximum possible metric distance between the entry point $x_{(1)}$ and any true answer x^* . This worst-case occurs when q lies strictly between $x_{(1)}$ and x^* (collinear).

Since G_0 is a t -spanner, the graph distance is bounded by t times the metric distance:

$$d_{G_0}(x_{(1)}, x^*) \leq t \cdot \text{dist}(x_{(1)}, x^*) \quad (4)$$

Combining these inequalities yields:

$$d_{G_0}(x_{(1)}, x^*) \leq t(\hat{d}_1 + \hat{d}_k) \quad (5)$$

As x^* was arbitrary, the inclusion holds for all true top- k points. $\square$

Figure 2 (left) illustrates Dijkstra's search region as the ball with radius $t \cdot (\hat{d}_1 + \hat{d}_k)$ centered at the first nearest neighbor. Any embedded vector outside of the ball is pruned.

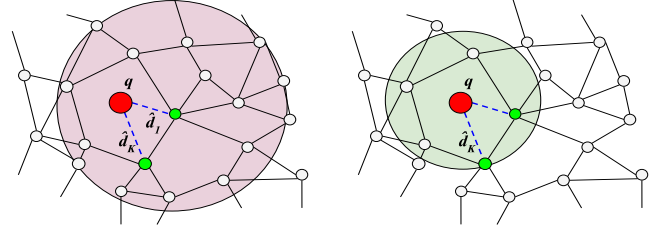


Figure 2: Stretch Bound Expansion. On the left, SBE-NN expands from the first NN of q a radius $\hat{d}_1 + \hat{d}_k$. On the right, SBE-Q expands a radius $\hat{d}_k$ from q

4.2 Stretch Bounded Expansion from Query (SBE-Q)

While SBE-NN (Section 4.1) guarantees exactness, the search radius relies on the triangle inequality to bridge the gap between the starting node $x_{(1)}$ and the query q . This introduces unnecessary “slack” in the search bound—specifically the term $\hat{d}_1 = \text{dist}(x_{(1)}, q)$. In high-dimensional spaces, even the nearest neighbor may be at a significant distance from q , inflating the search radius unnecessarily. To tighten this bound, we propose **SBE-Q**, which conceptually initiates the expansion directly from the query point q .

To formalize this, we treat the query q as a temporary node inserted into the graph G . Let $G' = (V', E')$ denote the graph augmented with q , where $V' = V \cup \{q\}$. Edges are established between q and the top- M_0 computed candidates to respect graph degree upper bound. (In the case $k < M_0$, we use k .) This virtual insertion alters the graph topology. In Section 4.3, we demonstrate how to efficiently test the stretch factor of G' . Let t' be the stretch factor of the augmented graph G' (which may differ from t). We derive a bound depending solely on $\hat{d}_k$ and t' , eliminating the $\hat{d}_1$ overhead (see Figure 2).

THEOREM 4.2 (STRETCH-BOUNDED EXPANSION FROM QUERY). *Let q be a query point virtually inserted into the graph G such that the augmented graph G' is a t' -spanner. Let $\hat{d}_k$ be the distance to the k -th candidate returned by HNSW. Every true top- k point lies inside the graph ball of radius $t' \cdot \hat{d}_k$ centered at q .*

Formally, for every $x^* \in N_k^*(q)$:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_k \quad (6)$$

PROOF. Let x^* be any arbitrary point in the true k -nearest neighbor set $N_k^*(q)$. By definition of the true set and the property that the HNSW k -th distance is an upper bound ($d_k^* \leq \hat{d}_k$):

$$\text{dist}(q, x^*) \leq d_k^* \leq \hat{d}_k \quad (7)$$

Since the augmented graph G' satisfies the t' -spanner property for the pair (q, x^*) , the shortest path distance in G' is bounded by the metric distance scaled by t' :

$$d_{G'}(q, x^*) \leq t' \cdot \text{dist}(q, x^*) \quad (8)$$

Combining Inequality (7) and Inequality (8), we obtain:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_k \quad (9)$$

Since this holds for any arbitrary $x^* \in N_k^*(q)$, the entire set of true nearest neighbors is contained within the graph ball $B_{G'}(q, t' \cdot \hat{d}_k)$. $\square$

Remark 1. The SBE-Q bound $d_{G'}(q, x) \leq t' \cdot \hat{d}_k$ is strictly tighter than the SBE-NN bound $d_{G_0}(x_{(1)}, x) \leq t(\hat{d}_k + \hat{d}_1)$ whenever $t' \approx t$. By shifting the expansion center to q , we remove the penalty $\hat{d}_1$ (the distance from the query to the closest returned candidate). In Section 7.4, we empirically demonstrate that this approach significantly reduces the volume of the graph explored while maintaining probabilistic guarantees.

Figure 2 (right) illustrates Dijkstra's search region as the ball with radius $t \cdot \hat{d}_k$ centered at the query point q . The Dijkstra's expansion is smaller than the expansion obtained by starting the search at the first nearest neighbor (left panel), because we do not need to traverse the additional $t \cdot \hat{d}_1$ term again.

4.3 Incremental Stretch Factor Estimation

For an HNSW graph $G = (V, E)$ of realistic size, re-evaluating the stretch factor distribution from scratch after every "virtual" query insertion is computationally prohibitive. We seek to maintain the validity of the EVT bounds derived in Section 3 without incurring the cost of a full resample.

When we simulate the insertion of a query node q , the population of vertex pairs expands. To maintain a representative sample of tail extremes without full re-computation, we employ a reservoir-style update that probabilistically injects new pairs involving q while retaining existing sample values.

Algorithm: Incremental Reservoir Tail Patching. Because the insertion of q adds edges to the graph, the shortest path distance $d_G(u, v)$ for any existing pair (u, v) can only decrease or remain constant; it cannot increase. Therefore, the previously sampled extreme stretch values either remain accurate or act as conservative overestimates. We define the update probability p as the likelihood that a uniformly random pair in the augmented graph involves the new node q . If a newly sampled stretch factor involving q falls into the extreme right tail, it is injected into the pool of block maxima, and the GEV parameters (μ, σ, ξ) are efficiently refitted. Algorithm 1 presents the overall approach.

4.3.1 Theoretical Guarantees. We assert that the sample of extreme values updated by the Incremental Reservoir Tail Patching algorithm allows for a valid certification of the maximum stretch factor using the methodology of Theorem 3.2.

THEOREM 4.3 (CONSERVATIVE EVT CERTIFICATION). *Let S_{ext} be the pool of extreme stretch values maintained during incremental insertion. Let $(\hat{\mu}, \hat{\sigma}, \hat{\xi})$ be the GEV parameters fitted to S_{ext} . For a confidence level β , the probability that the true maximum stretch factor of the augmented graph G' does not exceed the estimated return level t' is at least β .*

PROOF. The validity of this bound relies on the monotonically non-increasing nature of graph distances under edge addition. For retained pairs not involving q , the stored stretch value S^{old} satisfies $S^{old} \geq S^{true}$. Therefore, the right tail of our sampled distribution stochastically dominates the true right tail of the augmented graph.

Algorithm 1 Incremental Reservoir Tail Patching for Stochastic Stretch Estimation

Require: Existing sample S_N partitioned into m blocks of size n , Graph size N , New query node z , Target confidence β , Previous stretch bound t

Ensure: Updated sample S_{N+1} , Updated GEV parameters $(\hat{\mu}, \hat{\sigma}, \hat{\xi})$, Updated return level t'

```

1:  $p \leftarrow \frac{2}{N+1}$  {Probability a random pair involves  $z$ }
2:  $M_{update} \leftarrow \text{False}$  {Flag to track if any block maximum requires recalculation}
3: for  $i = 1$  to  $m$  do
4:   for  $j = 1$  to  $n$  do
5:     Draw  $u \sim \text{Uniform}(0, 1)$ 
6:     if  $u < p$  then
7:       Select random node  $v \in V_N$ 
8:       Compute  $d_G(u, z)$  and  $d_X(u, z)$ 
9:        $S_{i,j} \leftarrow \frac{d_G(u,z)}{d_X(u,z)}$  {Replace with new connection stretch}
10:       $M_{update} \leftarrow \text{True}$ 
11:     else
12:        $S_{i,j} \leftarrow S_{i,j}$  {Retain old conservative value}
13:     end if
14:   end for
15: end for
16:
17: if  $M_{update}$  is True then
18:   for  $i = 1$  to  $m$  do
19:      $M_{N+1,i} \leftarrow \max(S_{i,1}, \dots, S_{i,n})$ 
20:   end for
21:    $(\hat{\mu}, \hat{\sigma}, \hat{\xi}) \leftarrow \text{MLE}(M_{N+1})$  {Fit GEV distribution to the updated block maxima}
22:    $t' \leftarrow \hat{\mu} - \frac{\hat{\sigma}}{\hat{\xi}} \left[ 1 - (-\ln \beta)^{-\hat{\xi}} \right]$  {Compute new return level bound}
23: else
24:    $t' \leftarrow t$  {Retain previous bound  $t$  if reservoir was unchanged}
25: end if
26: return  $S_{N+1}, (\hat{\mu}, \hat{\sigma}, \hat{\xi}), t'$ 
```

Fitting the GEV distribution to these conservative maxima yields a return level t' that safely upper-bounds the true maximum stretch. $\square$

4.3.2 Complexity Analysis. The computational efficiency of this incremental strategy is superior to full resampling. The number of distance computations required follows a Binomial distribution $K \sim B(n, p)$, where $p = \frac{2}{N+1}$. The expected number of computations is $E[K] = \frac{2n}{N+1}$.

For a typical configuration where the graph size N is large, the expected graph distance sampling cost is negligible. The primary additional overhead is the Maximum Likelihood Estimation (MLE) required to update the three GEV parameters (μ, σ, ξ) . Because MLE is performed solely on the small subset of extreme tail values rather than the entire sample, it operates in $\mathcal{O}(m)$ time (where m is the number of block maxima), effectively allowing for continuous certification with near-zero marginal cost per simulated insertion.

5 METRIC BOUND VERIFICATION

While the Stretch-Bounded Expansion (SBE-Q and F-SBE-Q) described in Sections 4.2 and 8 provides a theoretical containment guarantee for the true k -nearest neighbors, naively evaluating the metric distance $\text{dist}(q, v)$ for every node v within the search radius $t' \cdot d_k$ can be computationally expensive. To bridge the gap between theoretical exactness and practical latency, we introduce Metric Bound Verification (MBV).

MBV is a pruning mechanism integrated directly into the graph traversal. It exploits the triangular inequality inherent in the metric space to discard candidates *within* the search radius without performing explicit distance computations. By maintaining recursive

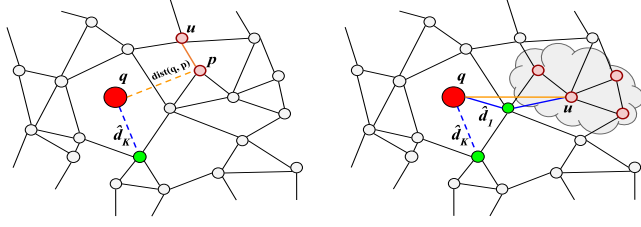


Figure 3: Metric Bounded Verification. On the left, Recursive Lower Bound Pruning is shown: the parent p was pruned, and the lower bound $LB(u)$ was then used to eliminate u . On the right, Elliptical Search Space Pruning is shown, where node u is pruned along with its descendants (Equation 11).

lower bounds on the distance from the query to the current node, we can rigorously certify that specific sub-branches of the graph cannot contain better candidates than those already found.

These pruning mechanisms are universal; they apply equally to the unconstrained SBE-Q search and the filtered F-SBE-Q search, as they rely solely on metric space properties independent of node attributes.

5.1 Recursive Lower Bound Pruning

The core of MBV relies on propagating lower bounds down the search tree (the path taken by Dijkstra’s algorithm from the virtual query node q). For any node u visited during the expansion, let p be its parent in the search trace (i.e., the node that added u to the priority queue). By the triangle inequality, the metric distance $dist(q, u)$ is lower-bounded by the distance to the parent minus the edge weight connecting them:

$$dist(q, u) \geq dist(q, p) - dist(p, u) \quad (10)$$

Since the edge weight $w(p, u)$ in the graph corresponds to the metric distance between p and u , we can derive a recursive lower bound $LB(u)$. This bound depends on whether the parent p was fully evaluated or merely pruned (where only a lower bound is known).

- **Evaluated Parent:** If $dist(q, p)$ has been computed (note if this distance was in the Search Trace $\mathcal{T}(q)$, as per section 2, it will be re-used), then: $LB(u) = dist(q, p) - w(p, u)$
- **Pruned Parent:** If p was pruned based on its own lower bound $LB(p)$, we propagate the uncertainty: $LB(u) = LB(p) - w(p, u)$

Let $\hat{d}_k$ be the distance to the current k -th nearest neighbor found so far (initially from HNSW, then updated dynamically). If $LB(u) > \hat{d}_k$, we can certify that u is strictly worse than the current candidate set and cannot improve the result. Consequently, we mark u as PRUNED and skip the expensive $dist(q, u)$ calculation (Figure 3). Crucially, we still explore u ’s neighbors, as the graph topology may lead back toward q , but we do so carrying the derived lower bound.

5.2 Elliptical Search Space Pruning

Beyond avoiding distance computations, we can also prune the search space itself using an “Elliptical Intersection” check. If the

exact distance $dist(q, u)$ is computed, we can determine if the entire branch stemming from u is effectively dead. A node u can be discarded—preventing the addition of its neighbors to the queue—if the path through u cannot possibly reach a point closer than the current k -th neighbor bound $\hat{d}_k$ within the remaining search budget.

Formally, if $d_{G'}(q, u)$ is the graph distance traveled so far and $R_{search} = t' \cdot \hat{d}_k$ is the current stretch-bounded radius, any descendant v must satisfy $d_{G'}(q, v) \leq R_{search}$. This implies the “remaining fuel” from u is $R_{search} - d_{G'}(q, u)$. If the metric distance $dist(q, u)$ exceeds this remaining capacity plus the target distance $\hat{d}_k$, no descendant can qualify. The pruning condition is (Figure 3):

$$d_{G'}(q, u) + dist(q, u) > t' \cdot \hat{d}_k + \hat{d}_k \quad (11)$$

This check tightens the search region from a simple graph ball to the intersection of the graph ball and the underlying metric constraints.

5.3 Algorithm

The integrated procedure, SBE-Q with Metric Bound Verification, is detailed in Algorithm 2. It replaces the naive expansion phases, ensuring that expensive metric evaluations are minimized while maintaining the probabilistic guarantees of exactness derived in Theorem 4.2.

This algorithm upgrades the heuristic search of an HNSW graph into an exact retrieval engine by rigorously bounding the search space and pruning unnecessary calculations. It operates through two integrated mechanisms:

Stretch-Bounded Expansion (SBE): The search treats the query q as a virtual node and expands strictly within a dynamic radius $R_{prune} = t' \cdot \hat{d}_k$. This radius is derived from the graph’s stretch factor t' and the current best-known k -th distance $\hat{d}_k$. Since the true k -th distance d_k^* satisfies $d_k^* \leq \hat{d}_k$, this radius guarantees that all true k -nearest neighbors are contained within the boundary. As better candidates are discovered, $\hat{d}_k$ decreases, automatically shrinking the search radius. Specifically:

Discovery of Better Candidates: As the expansion traverses the graph, it computes the exact metric distance d_{true} for a visited node u . If $d_{true} < \hat{d}_k$, u improves upon the current top- k set.

The Update Event: The algorithm inserts u into the priority queue of results (R_{topk}) and removes the previous furthest candidate. Consequently, $\hat{d}_k$ immediately decreases.

Tightening the Radius: This reduction in $\hat{d}_k$ triggers an immediate recalculation of $R_{prune} = t' \cdot \hat{d}_k$. The algorithm effectively “pulls in” the search horizon, pruning pending nodes in the queue that are now outside the new, tighter boundary.

Metric Bound Verification (MBV): To avoid expensive metric distance computations for every visited node, the algorithm applies two pruning checks:

- (1) **Recursive Lower Bound Pruning:** It utilizes the triangle inequality to propagate lower bounds from parent nodes to their children. If a node’s lower bound distance $LB(u)$ exceeds the current $\hat{d}_k$, the node is discarded without performing the expensive distance calculation.

- (2) **Elliptical Pruning:** It evaluates if a graph branch is “dead” by checking if the path through a node can possibly reach a better candidate within the remaining search budget. If $d_{G'}(q, u) + \text{dist}(q, u) > (t' + 1) \cdot \hat{d}_k$, the entire branch is pruned.

Together, these techniques guarantee the retrieval of the exact k -nearest neighbors while maintaining efficiency by aggressively filtering out unpromising candidates. Figure 3 (right) presents the Elliptical Search Space Pruning technique where the node u is far enough from q by graph distance (path shown in blue), such that any descendant of u (shown by the red cloud) in the shortest-path tree will not be within the top- k as they are far away in vector space as well (shown by the orange line) and thus, can be pruned.

6 STATISTICAL CERTIFICATION VIA CONFORMAL RISK CONTROL

We frame the “Certify” phase of our “Certify-then-Rectify” workflow as a distribution-free selective prediction problem. By leveraging Conformal Risk Control (CRC) [3, 4] and Learn-then-Test (LTT) [5] procedures, we construct a rigorous statistical certifier. This certifier acts as a gatekeeper, dynamically deciding whether to trust the fast, approximate candidates returned by HNSW, or to escalate to the exact, but computationally heavier, exact recovery procedures (SBE-Q and MBV).

6.1 Problem Setup and Certifiability Score

Let $\mathcal{N}_k(q)$ represent the approximate top- k candidates returned by the vanilla HNSW greedy traversal. Let $\mathcal{N}_k^*(q)$ represent the true k -nearest neighbors obtained via an exact oracle (in our framework, this oracle is SBE-Q coupled with MBV). We define the per-query recall as:

$$\text{Recall}(q) = \frac{|\mathcal{N}_k(q) \cap \mathcal{N}_k^*(q)|}{k} \quad (12)$$

Given a target recall threshold $\tau \in (0, 1]$, we call a top- k result set “compliant” to τ if $\text{Recall}(q) \geq \tau$. Given τ and a target risk level $\alpha \in (0, 1)$, our goal is to build a certifier $C(q)$ that outputs “accept” only when it can statistically guarantee that the top- k result set is compliant with τ .

To achieve this, we define a score $s(q)$ from HNSW’s internal search trace. We use a compact feature vector $z(q)$ that captures (i) distance geometry, (ii) search reach, and (iii) path stability:

$$z(q) = [\hat{d}_1, \dots, \hat{d}_{100}, \tilde{d}_1, \dots, \tilde{d}_{10}, |\mathcal{T}(q)|, n_{\text{rev}}(q), \Delta_{\text{rev}}(q)].$$

$\hat{d}_1, \dots, \hat{d}_{100}$ are the top-100 returned distances, and in the case $k < 100$, we pad with 0 entries. $\tilde{d}_1, \dots, \tilde{d}_{10}$ are the remaining top-10 candidate-frontier distances at termination. These features capture distance geometry. Following previous work [10], we capture the search reach with $|\mathcal{T}(q)|$, where $\mathcal{T}(q)$ is the search trace defined in Section 2. We use *distance-reversal* statistics for path stability: $n_{\text{rev}}(q)$ is the number of direction reversals in successive expansion distances, and $\Delta_{\text{rev}}(q)$ is their cumulative magnitude. More precisely, let

$$d_1^{\text{pop}}, d_2^{\text{pop}}, \dots, d_T^{\text{pop}}$$

denote the sequence of metric distances from q of the nodes popped from the bottom-layer HNSW candidate queue. A *distance reversal*

occurs at index $j \in \{2, \dots, T-1\}$ whenever the local direction of this sequence changes, i.e.,

$$(d_j^{\text{pop}} - d_{j-1}^{\text{pop}})(d_{j+1}^{\text{pop}} - d_j^{\text{pop}}) < 0.$$

Equivalently, the popped distances are locally non-monotone; for example, 100, 50, 120 and 50, 120, 80 each contain a reversal. Let $r_j(q)$ indicate whether a local reversal occurs at position j . We define $n_{\text{rev}}(q) = \sum_{j=2}^{T-1} r_j(q)$ as the number of reversals, and $\Delta_{\text{rev}}(q)$ as the sum of the adjacent jump magnitudes (ie. the two factors of the product in the above equation) over all positions j with $r_j(q) = 1$. These features quantify instability in the HNSW traversal order: larger values indicate that the search path is less geometrically consistent. Our feature-ablation results indicate that distance geometry contributes most of the score separability, while path stability statistics provide complementary gains in harder regimes: removing distance features reduces AUROC by 0.009 ~ 0.055 across datasets. Removing reversal features has a smaller average effect but still non-negligible (up to $\Delta \text{AUROC} = 0.017$). This supports using the full vector for cross-dataset robustness. The feature vector is then mapped through logistic regression trained on a calibration set. Crucially, the statistical guarantee is independent of the model’s predictive quality; a poor fit simply yields a more conservative certifier.

6.2 Guaranteeing Expected Shortfall via CRC

To guarantee the average quality of the certified results without making underlying distributional assumptions about the metric space, we utilize the Conformal Risk Control (CRC) framework. We define the recall shortfall as our per-query loss function:

$$l(q) = \max(0, \tau - \text{Recall}(q)) \quad (13)$$

Given a calibration set $\mathcal{D}_{\text{cal}} = \{(q_i, \mathcal{N}_k^*(q_i))\}_{i=1}^n$ drawn i.i.d. from an unknown query distribution $\mathcal{P}$, the procedure operates as follows:

- (1) Compute the score $s_i = s(q_i)$ and the loss l_i for each calibration query, sorting them in descending order of their score.
- (2) For any candidate threshold θ , define the empirical risk as:

$$\hat{R}(\theta) = \frac{\sum_{i=1}^n l_i \cdot \mathbb{1}[s_i \geq \theta]}{\sum_{i=1}^n \mathbb{1}[s_i \geq \theta] + 1} \quad (14)$$

- (3) Select the optimal threshold $\hat{\theta}$ that bounds this empirical risk:

$$\hat{\theta} = \inf \left\{ \theta : \hat{R}(\theta) \leq \alpha(1 - \tau) \cdot \frac{n}{n+1} \right\} \quad (15)$$

Theorem 6.1 (CRC Guarantee for HNSW Recall). *If the calibration queries $q_1, \dots, q_n$ are i.i.d. from $\mathcal{P}$, then for a new test query $Q \sim \mathcal{P}$, the certifier $C(Q) = \mathbb{1}[s(Q) \geq \hat{\theta}]$ satisfies:*

$$\mathbb{E}[l(Q) \cdot \mathbb{1}[s(Q) \geq \hat{\theta}]] \leq \alpha(1 - \tau) \quad (16)$$

This guarantee is exact in a finite-sample sense and requires no knowledge of HNSW’s internal workings.

6.3 Pointwise Guarantees via Learn-then-Test (LTT)

If a strict, per-query probabilistic guarantee is required, we deploy the Learn-then-Test (LTT) framework. LTT controls the certifier's precision, ensuring that $\mathbb{P}_{q \sim \mathcal{P}}[\text{Recall}(q) \geq \tau \mid s(q) \geq \hat{\theta}] \geq 1 - \epsilon$ with a confidence level of $1 - \alpha$.

We split the calibration set $\mathcal{D}_{\text{cal}}$ into a training set $\mathcal{D}_{\text{tr}}$ and a test set $\mathcal{D}_{\text{te}}$. From $\mathcal{D}_{\text{tr}}$, we define a finite set of candidate thresholds Θ . For each candidate threshold $\theta \in \Theta$ evaluated on $\mathcal{D}_{\text{te}}$, we test the null hypothesis $H_0^\theta : \mathbb{P}[\text{Recall}(q) < \tau \mid s(q) \geq \theta] > \epsilon$.

Let m_θ denote the number of certified queries and x_θ denote the subset of those queries that failed to meet the target recall. We compute the one-sided binomial p-value:

$$\text{p-value} = \sum_{j=0}^{x_\theta} \binom{m_\theta}{j} \epsilon^j (1 - \epsilon)^{m_\theta - j} \quad (17)$$

To account for multiple testing over Θ , we apply a Bonferroni correction, rejecting H_0^θ only if p-value $\leq \frac{\alpha}{|\Theta|}$. The final threshold is selected as:

$$\hat{\theta} = \max\{\theta \in \Theta : H_0^\theta \text{ is rejected}\} \quad (18)$$

6.4 End-to-End Guarantee

We certify two *different* events: the stochastic stretch estimation (SSE) underwrites *exactness* of the rectify branch with extreme-value confidence β , while the conformal certifier (CRC/LTT) controls τ -recall on the certify branch with a distribution-free guarantee. The end-to-end behaviour of CTR is therefore a mixture over the certify/escalate split, and the two guarantees live on independent probability spaces. We make this precise here.

Sources of randomness. We isolate three independent sources: (i) the calibration sample $\mathcal{D}_{\text{cal}} = \{(q_i, \mathcal{N}_k^*(q_i))\}_{i=1}^n \sim P^n$, which fixes the threshold $\hat{\theta}$; (ii) a fresh test query $Q \sim P$; and (iii) the reservoir/SSE sampling ω used to fit the augmented stretch factor t' on the query-augmented graph G'_Q , drawn only when Q is escalated.

Let $C(q) = \mathbf{1}[s(q) \geq \hat{\theta}]$ denote the certifier (Section ??), and let $\hat{N}(Q)$ be the CTR output: $\hat{N}(Q) = \mathcal{N}_k(Q)$ if $C(Q) = 1$ (certify), and $\hat{N}(Q) = \text{MBV}(Q)$ if $C(Q) = 0$ (escalate). Write $\rho := \Pr_Q[C(Q) = 1]$ for the certify rate induced by $\hat{\theta}$, and recall the per-query loss $\ell(q) = \max(0, \tau - \text{Recall}(q))$.

Assumptions.

Assumption 1 (Reachability). For every escalated query q , each true neighbor $x^* \in \mathcal{N}_k^*(q)$ is reachable from q in the augmented bottom-layer graph G'_q .

Assumption 2 (Calibration oracle). The ground-truth labels $\mathcal{N}_k^*(q_i)$ used to calibrate $\hat{\theta}$ are the true top- k sets.

Assumption 1 is the connectivity caveat of vanilla HNSW (cf. the rare *in degree* = 0 nodes); without it even exhaustive expansion within the stretch ball can miss a neighbor. Assumption 2 is discussed and relaxed in Remark 4.

LEMMA 6.1 (EXACTNESS OF THE RECTIFY BRANCH). Fix an escalated query q and let $\mathcal{E}_\omega$ be the event that G'_q satisfies the t'_ω -spanner

property on every pair (q, x^*) with $x^* \in \mathcal{N}_k^*(q)$. Under Assumption 1,

$$\Pr_\omega[\text{MBV}(q) = \mathcal{N}_k^*(q) \mid C(q) = 0] \geq \beta.$$

PROOF. By the conservative EVT certification (Theorem 4.3, via Theorem 3.2), the return level t'_ω upper-bounds the true maximum stretch of G'_q with confidence at least β , i.e. $\Pr_\omega[\mathcal{E}_\omega] \geq \beta$. Condition on $\mathcal{E}_\omega$. Theorem 4.2 then yields the containment

$$\mathcal{N}_k^*(q) \subseteq B_{G'_q}(q, t'_\omega \hat{d}_k),$$

so SBE- Q 's Dijkstra expansion, which explores exactly this graph ball, visits every true neighbor. The MBV pruning steps are lossless on the true top- k : a node u is discarded only when its certified lower bound satisfies $LB(u) > \hat{d}_k$, and since $\hat{d}_k \geq d_k^* \geq \text{dist}(q, x^*)$ for every $x^* \in \mathcal{N}_k^*(q)$ throughout the traversal, no true neighbor can ever trigger the pruning condition; the elliptical check is dominated by the same inequality. As $\hat{d}_k$ decreases monotonically toward d_k^* , the dynamic radius never drops below d_k^* . Hence, under Assumption 1, the procedure returns precisely $\mathcal{N}_k^*(q)$ on $\mathcal{E}_\omega$, and the claim follows. $\square$

Because the reservoir patch re-fits t' for each G'_q individually (Algorithm 1), Lemma 6.1 is per-query and transfers to $Q \sim P$ with no distributional assumption: $\Pr_{Q, \omega}[\text{Recall}(Q) = 1 \mid C(Q) = 0] \geq \beta$.

LEMMA 6.2 (CERTIFY BRANCH, RESTATED). With the score s , threshold $\hat{\theta}$, and loss ℓ of Section 6:

- (1) (CRC, exact, marginal over $\mathcal{D}_{\text{cal}} \times P$) $\mathbb{E}[\ell(Q) \mathbf{1}[C(Q) = 1]] \leq \alpha(1 - \tau)$.
- (2) (LTT; per-query, confidence $1 - \alpha$ over $\mathcal{D}_{\text{cal}}$) $\Pr_Q[\text{Recall}(Q) \geq \tau \mid C(Q) = 1] \geq 1 - \epsilon$.

THEOREM 6.3 (END-TO-END τ -COMPLIANCE OF CTR). Suppose Assumptions 1–2 hold.

- (1) **Per-query form (LTT route).** With probability at least $1 - \alpha$ over the draw of $\mathcal{D}_{\text{cal}}$,

$$\Pr_{Q, \omega}[\text{Recall}(Q) \geq \tau] \geq \rho(1 - \epsilon) + (1 - \rho)\beta \geq \min(1 - \epsilon, \beta).$$

- (2) **Expected-shortfall form (CRC route; unconditional, finite-sample exact).**

$$\mathbb{E}[\max(0, \tau - \text{Recall}(Q))] \leq \alpha(1 - \tau) + (1 - \beta)\tau,$$

$$\text{equivalently } \mathbb{E}[\text{Recall}(Q)] \geq \tau - \alpha(1 - \tau) - (1 - \beta)\tau.$$

PROOF. (i) Condition on the certify/escalate split, which is a function of $(Q, \hat{\theta})$ and is independent of the SSE randomness ω :

$$\Pr[\text{Recall}(Q) \geq \tau] = \rho \Pr[\text{Recall} \geq \tau \mid C=1] + (1 - \rho) \Pr[\text{Recall} \geq \tau \mid C=0].$$

On the event of probability $1 - \alpha$ over calibration, the first conditional is $\geq 1 - \epsilon$ by Lemma 6.2(2). The second is $\geq \beta$ by Lemma 6.1, since exact recovery gives $\text{Recall} = 1 \geq \tau$. The mixture is a convex combination of $1 - \epsilon$ and β , hence at least their minimum.

(ii) Decompose the expected loss by the certifier,

$$\mathbb{E}[\ell(Q)] = \underbrace{\mathbb{E}[\ell(Q) \mathbf{1}[C=1]]}_{\leq \alpha(1 - \tau) \text{ (Lemma 6.2(1))}} + \mathbb{E}[\ell(Q) \mathbf{1}[C=0]].$$

On the escalate branch $\ell(Q) = 0$ whenever the stretch event $\mathcal{E}_\omega$ holds (Lemma 6.1), and $\ell(Q) \leq \tau$ otherwise; as $\Pr_\omega[\neg \mathcal{E}_\omega] \leq 1 - \beta$

independently of the split, the second term is at most $\tau(1 - \beta)$. Summing yields the bound. $\square$

COROLLARY 6.4 (PROBABILITY OF IDENTIFYING THE TRUE k -NN). *Exact identification of the true k nearest neighbors corresponds to the compliance target $\tau = 1$. Under Assumptions 1–2, with probability at least $1 - \alpha$ over $\mathcal{D}_{\text{cal}}$,*

$$\Pr_{Q, \omega} [\hat{N}(Q) = \mathcal{N}_k^*(Q)] \geq \rho(1 - \epsilon) + (1 - \rho)\beta \geq \min(1 - \epsilon, \beta).$$

In particular, as $\epsilon \rightarrow 0$, $\alpha \rightarrow 0$, and $\beta \rightarrow 1$, the bound tends to 1: the probability of recovering the true k -NN can be made arbitrarily high.

PROOF. Set $\tau = 1$ in Theorem 6.3(i) and note that $\text{Recall}(Q) \geq 1$ is equivalent to $\hat{N}(Q) = \mathcal{N}_k^*(Q)$. $\square$

Remark 2 (Why $\tau = 1$ forces the LTT route). At $\tau = 1$ the CRC bound of Theorem 6.3(ii) degenerates to $\mathbb{E}[\ell(Q)] \leq 1 - \beta$: the conformal term $\alpha(1 - \tau)$ vanishes and *all* exactness flows from the EVT guarantee on the escalate branch. For the operating points used in our experiments ($\tau = 0.9$), certified queries return plain HNSW, which is only τ -good and carries *no* exactness guarantee; the honest unconditional bound on exact identification is then $\Pr[\hat{N}(Q) = \mathcal{N}_k^*(Q)] \geq (1 - \rho)\beta$ plus HNSW’s uncontrolled exact rate on the certified fraction. Thus Corollary 6.4 is a statement about the $\tau = 1$ regime; for $\tau < 1$ the meaningful end-to-end claim is the compliance guarantee of Theorem 6.3, not exactness.

Remark 3 (Two probability spaces). The confidence β is an extreme-value *return-level* guarantee over the stretch sampling ω , whereas $1 - \epsilon$ and $1 - \alpha$ are distribution-free conformal guarantees over P and $\mathcal{D}_{\text{cal}}$. The mixture in Theorem 6.3(i) is valid precisely because these sources are independent. They are not interchangeable: β inherits the usual GEV extrapolation optimism of a fitted return level, while the conformal quantities are finite-sample exact.

Remark 4 (Oracle-in-calibration coupling). The labels $\mathcal{N}_k^*(q_i)$ in $\mathcal{D}_{\text{cal}}$ are produced by the same MBV oracle, which is itself exact only with confidence β per query (Lemma 6.1). Consequently the conformal guarantees are, strictly, *recall-relative-to-oracle*. To make them recall-relative-to-truth one either assumes Assumption 2 directly, or pays a union bound over the calibration set, replacing $1 - \epsilon$ by $1 - \epsilon - n(1 - \beta)$ in Theorem 6.3(i). This is the single point at which the two phases are not cleanly independent.

6.5 The Query-Time Decision Rule

With $\hat{\theta}$ calibrated, the online query processing acts as a simple, two-step decision process:

Require: Query q

- 1: Run HNSW to obtain $\mathcal{N}_k(q)$ and compute the score $s(q)$.
- 2: **if** $s(q) \geq \hat{\theta}$ **then**
- 3: **CERTIFY:** Return $\mathcal{N}_k(q)$.
- 4: **else**
- 5: **ESCALATE:** Run the exact oracle (SBE-Q and MBV) and return $\mathcal{N}_k^*(q)$.
- 6: **end if**

7 EXPERIMENTS

7.1 Datasets and Implementation Setup

We augment `nmslib/hnswlib`² in C++ to implement the proposed CTR pipeline, including the rectification algorithm, namely MBV, on

²<https://github.com/nmslib/hnswlib>

Algorithm 2 Stretch-Bounded Expansion with MBV

Require: Query q , Graph G' , Stretch t' , Initial Candidates $\mathcal{H}_{\text{found}}$

Ensure: Exact k -nearest neighbor set $\mathcal{N}_k^*(q)$

```

1: Initialize:
2:  $R_{\text{top}K} \leftarrow$  Max-Priority Queue of size  $k$  (seeded with  $\mathcal{H}_{\text{found}}[: \min(M_0, k)]$ )
3:  $Q_{\text{visit}} \leftarrow$  Min-Priority Queue ordered by graph distance  $d_G$ 
4:  $\text{State} \leftarrow$  Map (node_id: {status, val})
5: Set initial bound based on HNSW results:
6:  $\hat{d}_k \leftarrow R_{\text{top}K}.\text{max\_dist}()$ 
7:  $R_{\text{prune}} \leftarrow t' \cdot \hat{d}_k$ 
8:  $Q_{\text{visit}}.\text{push}(0, q, \text{null}) \{(\text{distance}, \text{node}, \text{parent})\}$ 
9:  $\text{State}[q] \leftarrow \{\text{status: EVALUATED, val: 0}\}$ 
10: while  $Q_{\text{visit}}$  is not empty do
11:    $(d_{\text{graph}}, u, p) \leftarrow Q_{\text{visit}}.\text{pop}()$ 
12:   if  $d_{\text{graph}} > R_{\text{prune}}$  then
13:     break {Dynamic Termination}
14:   end if
15:   Calculate Lower Bound (MBV):
16:    $LB_u \leftarrow -\infty$ 
17:   if  $p \neq \text{null}$  then
18:      $w \leftarrow \text{edge\_weight}(p, u)$ 
19:      $LB_u \leftarrow \text{State}[p].\text{val} - w$  {Propagate distance or bound}
20:   end if
21:   Pruning Check & Metric Computation:
22:   if  $LB_u > \hat{d}_k$  then
23:      $\text{State}[u] \leftarrow \{\text{status: PRUNED, val: } LB_u\}$  {Skip expensive distance call}
24:   else
25:      $d_{\text{true}} \leftarrow \text{dist}(q, u)$  {Expensive computation}
26:      $\text{State}[u] \leftarrow \{\text{status: EVALUATED, val: } d_{\text{true}}\}$ 
27:     Update Result Set:
28:     if  $d_{\text{true}} < \hat{d}_k$  then
29:        $R_{\text{top}K}.\text{push}(d_{\text{true}}, u)$ 
30:        $\hat{d}_k \leftarrow R_{\text{top}K}.\text{max\_dist}()$ 
31:        $R_{\text{prune}} \leftarrow t' \cdot \hat{d}_k$  {Dynamically tighten search radius}
32:     end if
33:     Elliptical Pruning (Look-ahead):
34:     if  $d_{\text{graph}} + d_{\text{true}} > R_{\text{prune}} + \hat{d}_k$  then
35:       continue {Entire branch pruned}
36:     end if
37:   end if
38:   Expand Neighbors:
39:   for all neighbor  $v$  of  $u$  in  $G'$  do
40:     if  $v \notin \text{State}$  then
41:        $\text{new\_d} \leftarrow d_{\text{graph}} + \text{edge\_weight}(u, v)$ 
42:        $Q_{\text{visit}}.\text{push}(\text{new\_d}, v, u)$ 
43:     end if
44:   end for
45: end while
46: return  $R_{\text{top}K}$ 

```

Dataset	Description	d	metric	#queries
SIFT1M	Image feature transform vectors	128	l2	10k
GIST1M	GIST image descriptors	960	l2	1k
DEEP1M	Subset of deep image features	96	l2	10k
T2I10M	Subset of multimodal text-to-image search	200	cosine	5k
T2I100M	Subset of multimodal text-to-image search	200	cosine	5k

Table 1: Statistics Of Datasets Used

top of HNSW. We evaluate the system on three standard benchmark datasets: SIFT1M, GIST1M³, and DEEP1M, and two datasets for scalability test in Section 7.6: Text-to-Image (T2I) 10M and T2I 100M⁴ (vectors normalized). Dataset statistics are detailed in Table 1. All tests run with a single thread on an Intel Xeon Ice Lake 3.5 GHz processor with 247 GB memory and are memory-resident. For compilation, we use `-O3` and `-march=native`.

³Both SIFT1M and GIST1M from: <http://corpus-texmex.irisa.fr/>

⁴Both DEEP and T2I from: <https://research.yandex.com>

We vary M , ef_c (beam width during construction), ef_s , and k and examine both the cost of exact rectification and the behavior of the certification stage under different HNSW operating points. We therefore use a fixed 90% of queries provided by each dataset (the last column of Table 1) as the calibration set for conformal predictor training.

7.2 Validating Stretch Estimation via Level-2 Ground Truth

We validate our stochastic stretch estimation and characterize its sensitivity to key EVT hyperparameters using the level-2 HNSW subgraph, where the true maximum stretch t_{max} is computable by brute force over a tractable number of nodes⁵. We ablate the two parameters that must be committed to before sampling—confidence level β and block count m —across $(\beta, m) \in \{0.95, 0.995\} \times \{200, 400\}$ and vary sample size to generate four stretch value curves, using the brute-force t_{max} as ground truth. Test is conducted on SIFT, GIST, and DEEP with $M=32$, $ef_c=100$.

In Figure 4, we show that higher β consistently raises t^* since it selects a higher quantile of the fitted GEV and is more conservative. A larger block count ($m=400$) yields higher, more conservative estimates than $m=200$: more blocks supply more fitting points to the GEV, enabling better characterization of the higher t -value tail, whereas fewer blocks underfit the extremes. This comes at a convergence cost— $m=400$ saturates more slowly because smaller per-block samples produce more variable block maxima, so the fitted return level continues drifting as sample size grows rather than leveling off. Despite these differences, all (β, m) configurations yield a stable fitted t^* —one whose value no longer changes meaningfully with additional samples—by approximately 150k sampled pairs across all three datasets. For base-layer operation we use the conservative $m=400$, $\beta=0.995$ setting to mitigate stretch underestimation probability.

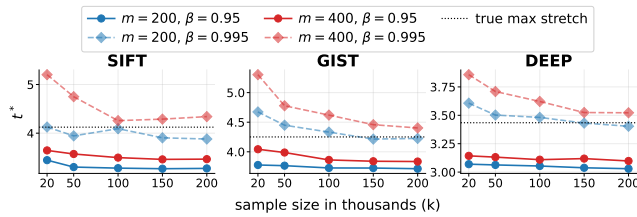


Figure 4: Stretch estimation ablating β and block count m .

7.3 Stretch and HNSW Construction Parameters

To further investigate how HNSW construction parameters influence the empirical maximum stretch of the resulting graph, we adopt the same Stochastic Stretch Estimation (SSE) procedure described in Section 7.2 and repeat it for graphs derived from various HNSW construction parameters. We report the estimated stretch across SIFT1M and DEEP1M, and use a parameter grid spanning $M \in \{16, 32, 48, 64\}$, $ef_c \in \{100, 200, 300\}$, with $\beta = 0.995$ fixed throughout. The results are presented in Figure 5.

⁵Vanilla HNSW has no guarantee in connectivity (see also [40]). Our stretch calculation is therefore conducted only for reachable nodes, despite unreachable nodes being rare (3 in degree=0 nodes in our SIFT1M $M=16$, $ef_c=200$ graph).

The graph shows that for fixed ef_c , increasing M consistently reduces the stretch, reflecting improved graph connectivity. Similarly, for fixed M , larger ef_c yields lower stretch, indicating higher-quality nearest-neighbor connections during construction. These findings align with the role of HNSW construction in shaping the graph's effective spanner properties and its associated stretch behavior.

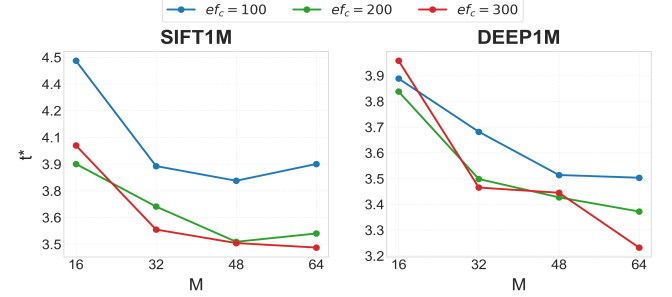


Figure 5: Effect of varying M and ef_c on stretch

7.4 System Ablation

We compare the number of distance computations (NDC) required to return the true k nearest neighbors across system variants. For ease of display, NDC is reported as a percentage of the total number of base nodes in the 1M-size datasets we use (NDC % of 1M), eg. 1% means we did 10k distance computations. Experiments fix $ef_s=100$ and vary $k \in \{10, 50, 100\}$. We evaluate four index configurations $(M, ef_c) \in \{16, 32\} \times \{100, 200\}$ on three datasets (SIFT1M, DEEP1M, GIST1M). This experiment is independent of the final certification stage and no target recall is enforced.

Experimental setup. In these experiments, we fix the underlying HNSW parameters to $M=32$, $ef_c=200$, and $ef_s=100$. Results are averaged over 100 random queries. In Figure 6, panels (a)–(c) show CRC and panels (d)–(f) show LTT on the three datasets. We fix the target recall threshold to $\tau=0.90$, and for LTT we set $\epsilon=0.5$. Each blue point is an operating point obtained by varying α , and the numeric annotation next to the point is the corresponding α value; runtime is reported on the x-axis as a multiple of plain HNSW under the same configuration. In Figure 7, we keep the same HNSW configuration and vary τ , with $\alpha=0.05$ and $\epsilon=0.5$ specifically for LTT; panels (a)–(c) show plain HNSW, panels (d)–(f) show CRC, and panels (g)–(i) show LTT.

Compared variants. We organise the variants into two families and compare five of them. The baselines, *SBE-NN* and *SBE-Q*, are expansion strategies that identify candidate nodes within the stretch-bounded radius without any pruning of final distance computations. Applying the methods lower-bound pruning (LBP) and elliptical pruning (EP) on top of *SBE-NN* yields *SBE-NN+LBP&EP*, and on *SBE-Q* gives the proposed *MBV*. To isolate the individual contribution of LBP and EP, we additionally include *SBE-Q+LBP*, which applies only the LBP component to *SBE-Q*; the gap between *SBE-Q+LBP* and *MBV* in Table 2 quantifies the saving attributable to EP.

Dataset	Metric k	$M = 16, efc = 100$			$M = 16, efc = 200$			$M = 32, efc = 100$			$M = 32, efc = 200$		
		10	50	100	10	50	100	10	50	100	10	50	100
SIFT1M	t	4.059	4.059	4.059	3.853	3.853	3.853	3.657	3.657	3.657	3.601	3.601	3.601
	SBE-NN	48.95	50.54	51.41	48.16	49.66	50.47	47.42	48.59	49.21	48.47	50.00	50.83
	SBE-NN+LBP&EP	40.07	42.31	43.20	39.95	43.14	44.61	39.30	42.24	43.54	40.67	43.84	45.32
	SBE-Q	5.60	14.07	18.66	4.04	11.38	15.82	3.40	10.13	14.22	3.62	10.37	14.67
	SBE-Q+LBP	4.01	11.19	15.54	2.88	8.95	12.99	2.45	7.85	11.60	2.57	8.23	12.11
DEEP1M	MBV	3.85	6.37	7.91	2.80	4.84	6.08	2.38	4.33	5.51	2.51	4.59	5.89
	t	4.251	4.251	4.251	4.148	4.148	4.148	4.067	4.067	4.067	3.867	3.867	3.867
	SBE-NN	81.03	84.75	86.52	71.36	76.12	78.40	83.43	86.86	88.51	83.08	86.53	88.19
	SBE-NN+LBP&EP	70.88	76.72	79.33	55.45	61.76	64.69	73.95	79.45	81.86	72.84	78.49	80.99
	SBE-Q	3.82	12.60	20.89	1.39	5.96	15.82	4.71	14.32	20.03	3.65	12.64	19.07
GIST1M	SBE-Q+LBP	3.48	10.04	17.23	1.22	4.35	13.29	4.12	11.49	16.09	3.08	9.51	14.72
	MBV	3.29	6.46	10.11	1.20	2.73	9.48	4.07	7.77	8.52	3.05	6.70	8.40
	t	4.470	4.470	4.470	4.119	4.119	4.119	3.930	3.930	3.930	3.646	3.646	3.646
	SBE-NN	84.91	86.08	86.63	83.83	85.15	85.77	85.10	86.40	87.01	83.95	85.26	85.87
	SBE-NN+LBP&EP	80.27	81.64	82.27	77.69	79.22	79.93	78.98	80.55	81.26	76.31	78.05	78.85
	SBE-Q	15.67	31.42	38.75	10.04	23.54	30.90	9.42	22.89	30.39	6.66	17.26	24.07
	SBE-Q+LBP	15.49	28.52	35.71	9.75	20.95	27.85	8.96	19.88	27.08	5.80	14.25	20.42
	MBV	15.01	23.62	26.04	9.70	16.55	18.58	8.94	15.58	17.52	5.64	11.47	13.15

Table 2: NDC as percentage of 1M base nodes across SIFT1M, DEEP1M, and GIST1M.

Results. Table 2 reveals two consistent trends. First, SBE-Q is a substantially stronger baseline than SBE-NN across all configurations, owing to its smaller expansion radius: eliminating the additive $\hat{d}_1$ term present in SBE-NN dramatically shrinks the graph region explored during rectification. Building on SBE-Q, applying LBP (SBE-Q+LBP) further reduces NDC by 14.3–26.4% across configurations relative to SBE-Q alone. Adding EP on top (MBV) achieves an additional 25.8–58.3% reduction across configurations by adaptively tightening the truncation radius as the running maximum of discovered k -NN distances decreases during Dijkstra traversal. Overall, the effect of augmenting SBE-Q with MBV is more pronounced in $k = 50$ or $k = 100$, as the larger d_k leads to a larger expansion radius, resulting in more nodes to explore and more pruning opportunities. MBV is consequently our best-performing method: on SIFT1M it reduces NDC to as low as 2.51% of base nodes at the finest index configuration for small k , and to around 5.6% on the higher-dimensionality and harder GIST1M dataset. Second, a cross-configuration trend mirrors the stretch estimation results: finer graphs (larger M or efc) yield smaller t^* , which in turn shrinks the expansion radius $t^* \hat{d}_k$ and lowers NDC for all five variants. Both trends above are consistent across different k .

7.5 Recall Target Compliance of Conformal Parameters

We next evaluate how the conformal parameters affect the final recall–runtime trade-off of CTR via the CRC/LTT. Rather than reporting average recall, we utilize per-query result-set compliance from Section 6.1 and report the *compliance rate* over a set of queries, which is the fraction of queries for which $Recall(q) \geq \tau$. Compliance rate is based on the result set after the rectification step, if certification fails.

In these experiments, we fix the underlying HNSW parameters to $M=32$, $efc=200$, and $efc_s=100$. In Figure 6, panels (a)–(c) show CRC and panels (d)–(f) show LTT on the three datasets. We fix the target recall threshold to $\tau=0.90$, and for LTT we set $\epsilon=0.5$. Each blue point is an operating point obtained by varying α , and the numeric annotation next to the point is the corresponding α value; runtime is reported on the x-axis as a multiple of plain HNSW

under the same configuration. In Figure 7, we keep the same HNSW configuration and vary τ , with $\alpha=0.05$ and $\epsilon=0.5$; panels (a)–(c) show plain HNSW, panels (d)–(f) show CRC, and panels (g)–(i) show LTT.

For both CRC and LTT, decreasing α makes certification stricter, so fewer HNSW outputs are certified directly and more queries are rectified. This raises the compliance rate but also increases runtime; larger α has the opposite effect. Across the τ -sweeps, the compliance rate of plain HNSW ($efc_s=100$) drops quickly as τ increases, whereas both CTR variants remain much flatter because rectification preserves compliance on hard queries. LTT achieves higher compliance at comparable runtime ratios. This improvement comes from LTT’s stricter per-query guarantee, which enables a more selective allocation of the heavier MBV computation to queries that are more likely to have low recall and therefore contribute the most to compliance gains. Take GIST for example, under the $\alpha = 5 \times 10^{-2}$ setting, LTT has four fewer false negatives and one fewer false positive than CRC in the runtime-comparable $\alpha = 0.3$ setting.

Notably, the small α values that appear in LTT are largely an inherent consequence of the test itself: the null benchmark is the boundary case with x_θ failures among m_θ certified queries under a Binomial(m_θ, ϵ) model, and the corresponding one-sided tail probability can become extremely small as soon as the observed failure count falls below the nominal level ϵm_θ . Thus, the practically useful range of α in LTT is naturally shifted toward smaller values; this is a property of the testing mechanism rather than an unusually aggressive confidence requirement.

Figure 8 further illustrates how the LTT threshold θ varies jointly with α and ϵ . The qualitative trend of θ varying with α and ϵ is consistent. In particular, along a fixed- ϵ slice of the surface, decreasing α increases the selected threshold θ . Since LTT certifies a query only when its score exceeds θ , a larger threshold makes certification more selective: fewer queries are accepted directly, more queries are sent to rectification, and the final compliance rate increases at the cost of higher latency. Conversely, for the same fixed ϵ , increasing α lowers θ , which certifies more queries directly and therefore reduces runtime, but also lowers compliance. To visually illustrate

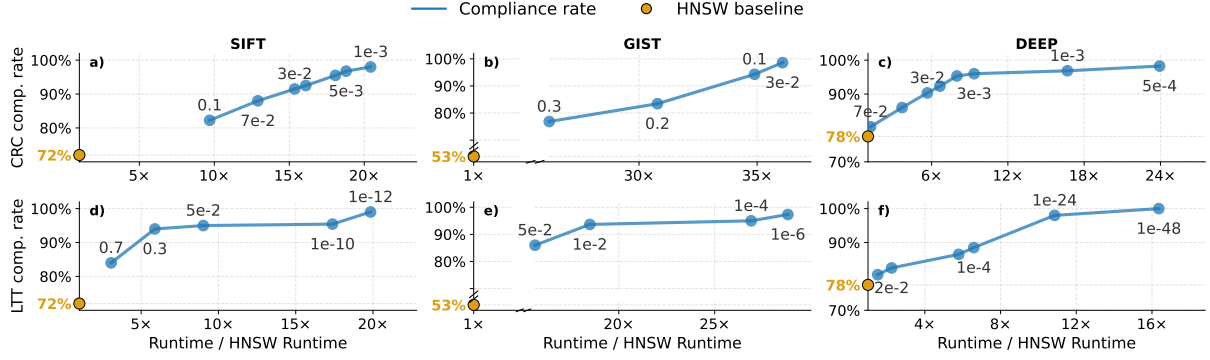


Figure 6: α -sweeps on three datasets. Blue line = compliance rate as α is varied; x-axis is in multiples of HNSW runtime.

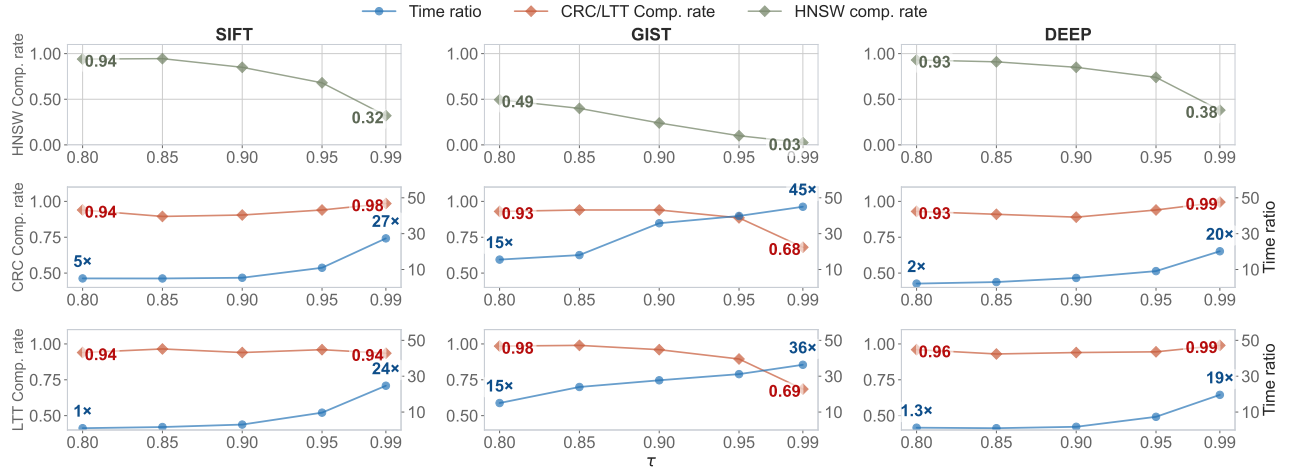


Figure 7: τ -sweeps on three datasets. Primary scale = compliance rate; secondary scale = time in multiples of HNSW runtime.

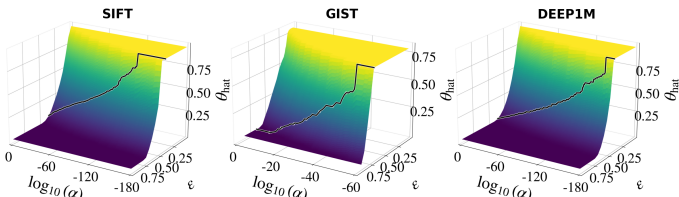


Figure 8: LTT threshold surface. Higher θ increases the fraction of queries sent to exact rectification. Black contours show fixed ϵ slices, highlighting how α controls the latency-compliance trade-off.

this trade-off, the black contour lines in Figure 8 depict constant- ϵ cross-sections of the threshold surface: $\epsilon=0.4$ for SIFT1M and DEEP1M, and $\epsilon=0.7$ for GIST1M. Each such cross-section isolates the one-dimensional family of operating points obtained by varying α while holding ϵ fixed, thereby making clear how the resulting change in θ governs the latency-compliance trade-off described above. The figure also suggests a practical tuning strategy: first select ϵ so as to avoid the broad plateau region of θ (the yellow region, where variation in ϵ has relatively little effect on the threshold), and then vary α to choose the desired operating point along the corresponding constant- ϵ trade-off curve.

We next study how the HNSW operating point affects the latency-compliance trade-off. In Figure 9, blue/green use $M=16$ and red/brown use $M=32$; blue/red use $ef_s=200$, while green/brown use $ef_s=300$. Panels (a)–(c) show CRC and panels (d)–(f) show LTT for SIFT, GIST, and DEEP. Across the rerun, increasing ef_s generally shifts curves upward and left, and increasing M produces a similar improvement in these operating ranges. The reason is that both changes improve the quality of the initial HNSW search: larger ef_s explores more candidates at query time, while larger M gives the graph more connectivity. As a result, more queries already meet the compliance target before rectification, so the expensive recovery step is triggered less often. This lowers the runtime ratio needed to reach a given compliance rate. Overall, Figure 9 suggests that ef_s is the cleaner query-time knob, while M provides a stronger index-level operating point when its memory and build-time cost is acceptable.

The CTR framework naturally extends to filtered search on top of algorithms such as ACORN [40]. We present our results in Section 8.2.

7.6 Scalability Experiments

A natural concern is whether dataset size adversely affects the runtime overhead of CTR: larger datasets contain more candidate nodes within any search radius, which could inflate the cost of

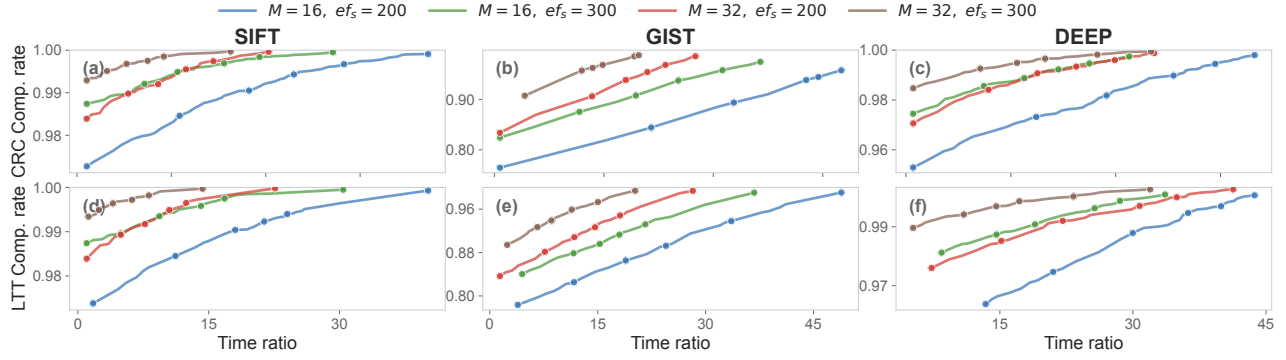


Figure 9: HNSW-parameter ablation on three datasets. X-axis reports runtime as a multiple of HNSW runtime.

SBE-Q. We test this using the Yandex Text-to-Image dataset⁶ at two scales—T2I-10M and T2I-100M—with index configuration $M=32$, $ef_c=200$ on normalized vectors, with an Intel Xeon Ice Lake 3.5 GHz processor with 64GB memory. For each scale, stretch estimation is carried out following the same procedure as Section 7.2 ($\beta=0.995$, $m=400$). As we use a subset of T2I-1B, query ground truths were re-computed by brute-force.

	T2I-10M ($t=4.133$)			T2I-100M ($t=4.124$)		
k	10	50	100	10	50	100
million NDC	0.55	1.16	1.55	2.33	4.63	6.11
% of N	5.55	11.64	15.51	2.33	4.63	6.11

Table 3: MBV NDC for $k \in \{10, 50, 100\}$.

We first run MBV without the certifier on 200 text queries sampled from T2I. Table 3 reports average NDC in absolute value and as a fraction of dataset size. Stretch estimation on 10M and 100M scales yields $t=4.133$ and $t=4.124$ respectively, confirming that the graph stretch does not grow appreciably with dataset size. At the same time, a $10\times$ larger corpus means base vectors are distributed more densely in the ambient space: the k -th true neighbor of a typical query lies closer, so d_k decreases. Since the SBE-Q search radius equals $t \cdot d_k$, a smaller d_k directly shrinks the expansion ball, and that ball covers a smaller fraction of the N base nodes. Table 3 reflects this exactly: the expansion fraction drops by roughly $2.5\times$ across all k values when moving from 10M to 100M (eg. $k=10$: $5.55\% \rightarrow 2.33\%$). Recall is consistently 1 in all settings.

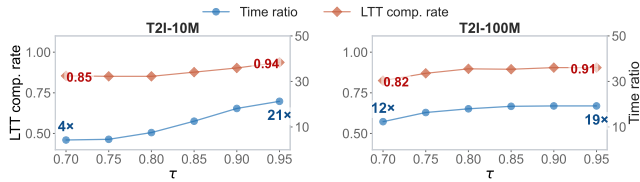


Figure 10: τ ablation on T2I-10M and T2I-100M.

We next conduct a full τ -sweep ($ef_s=100$, $k=100$) to evaluate end-to-end overhead. Absolute latency increases with dataset size, as expected: HNSW grows by $8\text{--}9.4\times$ and CTR by $7.5\text{--}8.4\times$. The key

question is whether CTR adds disproportionate overhead beyond this baseline HNSW growth. Figure 10 shows that the normalized runtime ratio remains comparable across scales: the peak ratio even decreases from $21.3\times$ at 10M to $19.2\times$ at 100M. Thus, because the expansion radius remains bounded, CTR preserves the compliance–runtime trade-off across scales and introduces no additional scaling bottleneck beyond HNSW itself.

7.7 ConANN vs. CTR

We now perform an *end-to-end* system comparison against ConANN [25], a recent ANN system that calibrates a conformal early-termination rule on an IVF index to also control approximation error and meet recall target. Both systems are evaluated on three datasets (SIFT1M, DEEP1M, and T2I-10M with normalized vectors), sweeping the recall target $\tau \in \{0.80, 0.85, 0.90, 0.95, 0.99\}$ (points connected in order of increasing τ values on graph) with $k=100$; we report the per-query compliance rate $\Pr[\text{Recall}(q) \geq \tau]$ against end-to-end throughput.

Index configurations. ConANN indexes each dataset with IVF-Flat using 1024 Voronoi cells and exact L_2 scoring within probed cells, α calculated as $1 - \tau$. CTR runs on HNSW with $M=32$, $ef_c=200$, and query-time $ef_s=100$; we certify each result with LTT (Section 6) and rectify queries that fail certification with MBV.

Results. Figure 11 reports the compliance–throughput trade-off. Across all three datasets CTR attains higher compliance than ConANN at every comparable target; the systems differ chiefly in where they sit on the throughput axis. On SIFT1M, CTR is throughput-competitive: at modest targets ($\tau \leq 0.90$) it is as fast or faster than ConANN—at $\tau=0.85$ LTT sustains 1.7k QPS at 91% compliance versus ConANN’s 1.5k QPS at 67%—and only at stringent targets does ConANN regain a throughput edge (e.g. 789 versus 97 QPS at $\tau=0.95$), where its compliance nonetheless plateaus at 0.72–0.82 against CTR’s 0.94–0.99. The gap in compliance rate is a direct consequence of the certify-then-rectify mechanism: LTT supplies a per-query confidence guarantee, and this guarantee, combined with CTR’s ability to recover the exact nearest neighbors for queries flagged as low-quality, is what gives CTR its edge, lifting the recall of such queries toward 1 and raising the overall compliance rate. ConANN, by contrast, can only reorder and truncate its IVF probe and cannot exactly recover the neighbors it misses, so a fixed fraction of queries necessarily remains below τ at every point.

⁶<https://research.yandex.com/blog/benchmarks-for-billion-scale-similarity-search>

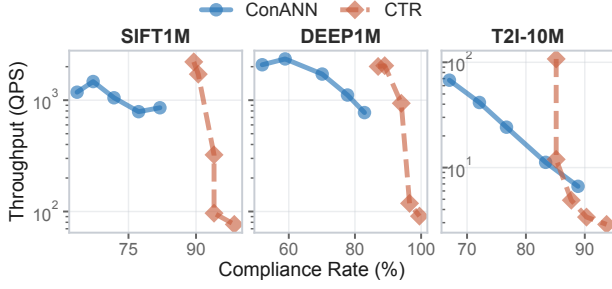


Figure 11: ConANN vs. CTR (upper-right is better).

7.8 DARTH vs. CRC/LTT

Recently, DARTH [12] was proposed solely as a predictor of recall. We therefore compare it directly against CRC and LTT in a certifier-only setting, i.e., without true neighbor rectification (as DARTH does not have this capability). Table 4 reports the F1 scores with respect to the binary event $Recall(q) \geq \tau$. For fairness, we use DARTH only as a recall predictor and let HNSW run to the same stopping point in all methods, rather than using DARTH’s early-termination mechanism. The experiment is conducted on SIFT1M with $M=32$, $ef_c=200$, $ef_s=100$, $k=100$ and 1000 random queries, utilizing the implementation from the DARTH authors. Across all three target levels, CRC/LTT attain higher F1, with the gap becoming larger at stricter targets. Similar results were obtained across all data sets and are omitted for brevity. DARTH is a best-effort predictor and does not formally control the fraction of queries whose recall falls below the target. In contrast, CRC and LTT are calibrated for fixed deployment settings (dataset, index configuration, workload, and target recall τ) and explicitly control compliance-related errors under that setting. Consequently, for deciding whether an HNSW result should be accepted as meeting a prescribed recall target, CRC/LTT yield a more reliable acceptance rule.

τ	DARTH		CTR	
	Default	Tuned	CRC	LTT
0.85	0.968	0.971	0.983	0.983
0.90	0.913	0.913	0.965	0.966
0.95	0.709	0.709	0.865	0.874

Table 4: Certifier comparison on SIFT1M: F1 Scores

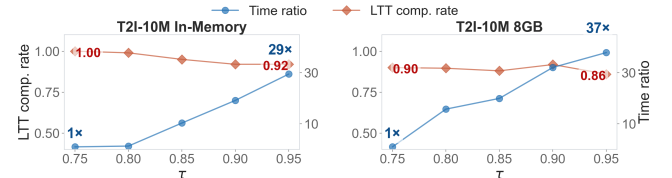
7.9 Extension to Disk-based Search

In real production settings, loading the full vector index into memory is often infeasible at billion-scale. Vector search methods that account for disk I/O are therefore necessary. In this section we show that CTR extends to such settings using DiskANN [27], which achieves state-of-the-art disk-based ANN performance by storing a Vamana graph index on disk and exploiting beam-search with asynchronous I/O to amortise latency across concurrent node fetches. **Experimental Setup.** We consider three settings. (1) **In-memory:** the Vamana graph variant of DiskANN, where the full graph with full-precision vectors is loaded into memory prior to ANN search. We use an Intel Ice Lake node with 64GB DDR4 unified memory. (2) **Limited memory:** memory is insufficient to hold the entire database. (3) We additionally increase result size k to assess the

impact on MBV expansion radius based on the limited memory setting, which scales with $t \cdot d_k$. This configuration stress-tests MBV under heavier disk-I/O.

DiskANN search retains only quantized vectors and graph edges in memory while exact vectors reside on disk. To preserve the exactness guarantees of our algorithm, we forgo the quantized vectors and instead keep precomputed graph edge weights in memory, which costs ~ 2.6 GB on the T2I-10M graph ($R=64$). The MBV algorithm is otherwise unchanged: it performs Dijkstra expansion using solely these in-memory graph edges; the only difference is that exact distance computations (Algorithm 2, line 25) issue disk reads on cache misses. In this setting, we use an Intel Ice Lake node with 8GB DDR4 memory attached to a 75GB local NVMe drive. Experiments use the T2I-10M dataset; when fully loaded, it requires ~ 8 GB for base vectors and ~ 2.6 GB for the graph ($R=64$), totalling ~ 10.6 GB—exceeding the 8GB constraint but well within the 64GB limit of Setting 1. For all settings we run a τ -sweep following Section 7.5, using LTT for all conformal predictors. Settings (1) and (2) results in the left and right panels of Figure 12 respectively; setting (3) results in Figure 13.

Parameters. We evaluate setting 1, 2 with $k = 10$ and setting 3 with $k = 20$. We use $\alpha = 0.1$, $\varepsilon = 0.5$, and $\tau \in \{0.75, 0.80, 0.85, 0.90, 0.95\}$. The DiskANN index is built with maximum degree $R = 64$ and construction list size $L_{\text{build}} = 100$; search uses list size $L = 100$ and disk beam width $B = 2$. MBV is initialised from a single seed with 10,000 hot-cached nodes in Vamana mode. Disk reads are issued via `io_uring` in basic single-issuer mode with direct I/O and a submit-batch size of 8, using a single thread. The MBV graph stretch parameter $t = 3.71$ is estimated following Section 7.4.

Figure 12: $k=10$: in-memory Vamana vs. DiskANN.

Results. As shown in Figure 12, the in-memory Vamana setting (left panel) confirms that CTR generalises beyond HNSW: it extends directly to the Vamana graph and achieves compliance–runtime trade-offs comparable to those observed on HNSW in Section 7.5. In the limited-memory setting with $k=10$ (right panel of Figure 12), the additional disk I/O incurred by MBV expansion remains controlled, adding 8–10 $\times$ DiskANN time to the runtime ratio at each τ level. Notably, compliance rates in the limited-memory setting are consistently lower than in the in-memory setting. This gap arises because native DiskANN beam search relies on quantized vectors as distance proxies, yielding a weaker ANN baseline than exact Vamana search; as a result, the conformal certifier operates on lower-quality candidates, reducing the fraction of queries that pass certification.

Figure 13 shows the effect of increasing k from 10 to 20 (setting (3)), which enlarges the MBV search radius ($t \cdot d_k$ grows as d_k increases with k), expanding the explored graph fraction by approximately 70% for queries that fail certification and thereby increasing

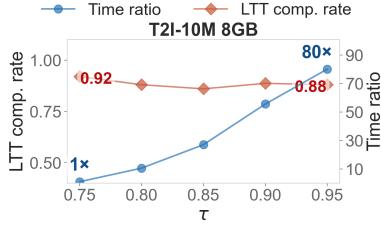


Figure 13: τ -sweep, $k=20$: disk-based DiskANN (8GB + NVMe).

disk I/O. This is directly reflected in the runtime ratio: at $\tau=0.9$, the ratio rises from approximately $30\times$ to $50\times$ the DiskANN baseline. Overall, MBV transfers to disk-based vector search without any algorithmic modification.

8 FILTERED STRETCH-BOUNDED EXPANSION FROM QUERY (F-SBE-Q)

In this Section, we show the CTR methodology can be directly extended to handle filtered ANN search (FANNS), and provide experimental results of our implementation. In FANNS, the goal is to retrieve the approximate k -nearest neighbors that satisfy a specific predicate condition P . Standard heuristic traversals often fail in these scenarios when filters disconnect the “small world” topology. However, by leveraging the global connectivity of the underlying graph structure—regardless of the predicate status of intermediate nodes—we can guarantee exact recovery of true nearest neighbors using the bounds derived from the approximate search results.

8.0.1 Problem Formulation. Let $P : \mathcal{X} \rightarrow \{0, 1\}$ be a boolean predicate function. We seek the set of k nearest neighbors $\mathcal{N}_{k,P}^*(q)$ such that for all $x \in \mathcal{N}_{k,P}^*(q)$, $P(x) = 1$. Let $d_{k,P}^*$ denote the distance to the true k -th nearest neighbor in the satisfying set:

$$d_{k,P}^* = \max_{x \in \mathcal{N}_{k,P}^*(q)} \text{dist}(q, x) \quad (19)$$

Let $\mathcal{H}_{\text{found}}$ be the set of candidates returned by the initial HNSW search (or any heuristic phase). We identify the subset of these candidates that satisfy the predicate P . If at least k such candidates exist, let $\hat{d}_{k,P}$ denote the distance to the k -th closest satisfying candidate found so far.

Similar to the unfiltered case, the distance to the k -th candidate found by any approximate method is an upper bound on the true k -th distance:

$$d_{k,P}^* \leq \hat{d}_{k,P} \quad (20)$$

8.0.2 Theoretical Guarantees. We extend Theorem 4.2 to the filtered case using the observable bound $\hat{d}_{k,P}$. The critical insight is that while the *targets* must satisfy P , the *path* taken in the graph G' to reach them need not. Thus, we rely on the spanner property of the full augmented graph G' .

THEOREM 8.1 (FILTERED SBE-Q). *Let q be a query point virtually inserted into the graph G such that the augmented graph G' is a t' -spanner. Let $\hat{d}_{k,P}$ be the distance to the k -th approximate neighbor satisfying predicate P (as returned by HNSW or updated dynamically). Then, every true filtered top- k point lies inside the graph ball of radius $t' \cdot \hat{d}_{k,P}$ centered at q .*

Formally, for every $x^* \in \mathcal{N}_{k,P}^*(q)$:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_{k,P} \quad (21)$$

PROOF. Let x^* be any arbitrary point in the true filtered set $\mathcal{N}_{k,P}^*(q)$. By definition of the true set and the upper bound property of the approximate results:

$$\text{dist}(q, x^*) \leq d_{k,P}^* \leq \hat{d}_{k,P} \quad (22)$$

Since the augmented graph G' is a t' -spanner over the full set of vertices $V \cup \{q\}$, there exists a path between q and x^* in G' such that:

$$d_{G'}(q, x^*) \leq t' \cdot \text{dist}(q, x^*) \quad (23)$$

Substituting the metric bound:

$$d_{G'}(q, x^*) \leq t' \cdot \hat{d}_{k,P} \quad (24)$$

Thus, the search radius defined by the approximate results is sufficient to encompass all true answers. $\square$

8.0.3 Algorithm. The expansion algorithm (Algorithm 3) proceeds by traversing the graph G' starting from q . Crucially, to preserve the spanner property t' , the traversal visits nodes regardless of whether they satisfy P . The predicate $P(v)$ is checked only during the candidate collection phase to update the dynamic bound $\hat{d}_{k,P}$. While Algorithm 3 presents the logic for predicate handling, in practice, this traversal is further optimized using the Metric Bound Verification techniques we introduce in Section 5.

8.1 Experiments On Stretch-Bounded Expansion For Filtered Search

To demonstrate SBE extends naturally to filtered ANN search (FANNS), we augment ACORN [40], a representative filtered-search method, with SBE-Q. Our experiments in this section are comparable to the setup in Section 7.4. Note that ACORN is a variant of HNSW that adds extra edges to the underlying graph, controlled by a parameter γ . In essence, the higher the selectivity of the query predicates (fewer matching vectors), the higher the value of γ required, which produces denser graphs that keep the search traversal highly navigable. Moreover, ACORN explores 2-hop neighbors at each step to maximize the probability of retrieving nearest neighbors that satisfy the query predicates.

In order to simulate the impact of predicate selectivity on our framework, we introduce a selectivity parameter σ , which we define as the percentage of database vectors satisfying a query predicate. For instance, the value $\sigma = 1$ would imply that only 1% of vectors pass the filter (highly restrictive), while $\sigma = 25$ admits 25% (moderately selective). In our experiments, we set the value of γ as the inverse of the selectivity parameter σ . We evaluate the performance of SBE-NN, SBE-Q, and MBV on ACORN across three datasets (SIFT1M, DEEP1M, and GIST1M respectively). Observe that the results in Table 5 depict trends similar to those observed for HNSW (Table 2).

As shown in Table 5, decreasing σ (i.e., higher selectivity) generally increases the number of nodes explored by SBE-Q, subject to the estimated stretch and density of the underlying graph. When either the density or stretch of the graph is high, the number of nodes explored by SBE-Q also increases, while guaranteeing perfect recall across all cases (subject to the estimation of the stretch

Dataset	Metric σ	$M=16, M_\beta=32, ef_c=200, k=10$				$M=16, M_\beta=32, ef_c=200, k=100$				$M=32, M_\beta=64, ef_c=200, k=10$				$M=32, M_\beta=64, ef_c=200, k=100$			
		1	5	10	25	1	5	10	25	1	5	10	25	1	5	10	25
SIFT1M	t (SSE c)	4.699	4.203	4.175	6.089	4.699	4.203	4.175	6.089	4.052	3.775	3.750	3.641	4.052	3.775	3.750	3.641
	SBE-NN (% of 1M)	92.06	82.14	82.39	85.15	97.09	89.60	89.24	87.23	85.71	74.25	74.74	68.64	92.58	82.66	82.69	76.89
	SBE-Q (% of 1M)	28.72	15.31	12.64	25.50	44.35	32.86	29.25	41.96	28.16	14.94	12.28	8.30	45.38	32.68	28.97	22.67
	MBV (% of 1M)	28.48	15.10	12.44	25.01	44.15	32.60	28.91	40.35	27.97	14.75	12.11	8.18	45.24	32.48	28.76	22.46
DEEP1M	t (SSE c)	4.369	3.946	3.914	6.628	4.369	3.946	3.914	6.628	3.571	3.560	3.520	3.608	3.571	3.560	3.520	3.608
	SBE-NN (% of 1M)	98.81	93.94	92.92	91.29	99.77	96.92	96.27	91.47	97.93	95.40	94.14	93.25	99.47	97.73	96.89	96.21
	SBE-Q (% of 1M)	21.94	7.99	6.55	45.76	60.88	30.21	24.61	72.09	13.97	8.48	6.17	6.04	51.15	34.38	26.51	24.04
	MBV (% of 1M)	21.91	7.97	6.54	45.60	60.85	30.17	24.58	72.00	13.96	8.47	6.16	6.03	51.12	34.35	26.48	24.00
GIST1M	t (SSE c)	5.207	5.199	5.180	6.984	5.207	5.199	5.180	6.984	3.887	3.387	3.387	3.909	3.887	3.387	3.387	3.909
	SBE-NN (% of 1M)	90.70	89.98	90.12	61.63	91.77	90.74	90.81	61.83	93.66	87.88	87.51	91.47	94.87	89.50	89.05	92.56
	SBE-Q (% of 1M)	53.08	49.25	47.25	45.22	69.05	64.61	62.64	51.62	37.33	15.38	13.74	27.85	60.83	35.63	32.61	48.30
	MBV (% of 1M)	52.99	49.12	47.08	45.21	69.00	64.55	62.58	51.61	36.99	15.13	13.49	27.40	60.68	35.33	32.30	47.97

Table 5: Expanded-node coverage (% of 1M) on ACORN: SBE-NN vs SBE-Q vs MBV

Algorithm 3 Filtered Stretch-Bounded Expansion from Query (F-SBE-Q)

Require: Query q , Predicate P , Graph G' , Stretch t' , Initial Candidates $\mathcal{H}_{found}$
Ensure: Exact filtered set $\mathcal{N}_{k,P}^*(q)$

```

1: Initialize:
2:  $Q_{visit} \leftarrow$  Min-Priority Queue ordered by graph distance  $d_{G'}$ 
3:  $R_{topK} \leftarrow$  Max-Priority Queue of size  $k$  (stores pairs  $(dist(q, u), u)$  where  $P(u) = 1$ )
4:  $Visited \leftarrow \{q\}$ 
5: {Seed with initial heuristic results that satisfy  $P$  to establish  $\hat{d}_{k,P}$ }
6: for  $u \in \mathcal{H}_{found}$  do
7:   if  $P(u)$  is True then
8:      $R_{topK}.push(dist(q, u), u)$ 
9:   end if
10: end for
11:  $Q_{visit}.push(0, q)$ 
12: while  $Q_{visit}$  is not empty do
13:    $(d_{graph}, u) \leftarrow Q_{visit}.pop()$ 
14:   {Dynamic Termination Condition using  $\hat{d}_{k,P}$ }
15:    $\hat{d}_{k,P} \leftarrow R_{topK}.max\_dist()$  {Returns  $\infty$  if  $< k$  found}
16:   if  $d_{graph} > t' \cdot \hat{d}_{k,P}$  then
17:     break {Guaranteed to have found all true filtered neighbors}
18:   end if
19:   {Expand Neighbors}
20:   for each neighbor  $v$  of  $u$  in  $G'$  do
21:     if  $v \notin Visited$  then
22:        $Visited.add(v)$ 
23:        $new\_d_{graph} \leftarrow d_{graph} + dist(u, v)$ 
24:       {Check Metric Distance & Predicate}
25:        $d_{metric} \leftarrow dist(q, v)$ 
26:       if  $P(v)$  is True and  $d_{metric} < R_{topK}.max\_dist()$  then
27:          $R_{topK}.push(d_{metric}, v)$  {Updates  $\hat{d}_{k,P}$  (tightens bound)}
28:       end if
29:        $Q_{visit}.push(new\_d_{graph}, v)$ 
30:     end if
31:   end for
32: end while
33: return  $R_{topK}$ 

```

t , which varies with the parameter γ). We typically observe this increase because more selective predicates both raise the density of the underlying graph (through parameter γ) and yield a larger d_k ; that is, the top- k filtered results lie farther from q , which in turn enlarges the search radius. Our results further demonstrate the benefits of SBE-Q and MBV over SBE-NN, consistent with what we observed for HNSW. It is known [40] that ACORN exhibits low recall for highly selective predicates, and our framework resolves this.

8.2 Conformal Parameter Evaluation on Filtered Search

This subsection presents extended results demonstrating that our end-to-end Certify-then-Rectify (CTR) framework generalizes to the FANNS setting. Here the experiment is comparable to Section 7.5 in its motivation. In the unfiltered case, the previous sections establish that wrapping vanilla HNSW with our statistical certifiers (CRC and LTT) and exact recovery procedures (SBE-Q with MBV) yields a principled recall-runtime trade-off: queries whose HNSW results are statistically certified as meeting the target recall τ are returned directly, while those that fail certification are escalated to exact recovery. Here, we investigate whether the same framework transfers to *filtered* search, where the goal is to retrieve the k nearest neighbors that additionally satisfy a boolean predicate P . Our results span a total of six datasets. We use the SIFT1M, GIST1M and DEEP1M datasets where we introduce synthetic predicates - in essence, given a value σ , we randomly select σ percent of vectors to satisfy a boolean predicate. We further evaluate our framework on three datasets with real predicates. This include the PAPER and LAION1M datasets from the ACORN paper, moreover, to demonstrate scalability, we evaluate our framework on the Arxiv dataset, which contains high dimensional vectors.

8.2.1 Experimental Setup. Across all three datasets, we used the same index and search settings: $ef_c=64$, $ef_s=128$, and $k=50$, with $M=16$ and the compression factor $M_\beta=32$. The LTT parameter ϵ was dataset-dependent in the α -sweep experiments: we used $\epsilon=0.8$ for GIST and $\epsilon=0.5$ for DEEP1M and SIFT1M. Separately, in the θ -surface plots, we swept ϵ over the range 0.3 to 0.9. The certifier's α grids and τ values were also chosen per preset; for example, the τ -sweeps used 0.65–0.75 for GIST and 0.82–0.90 for DEEP. Throughout these experiments, the core HNSW settings $ef_c/ef_s/k$ remained fixed at 64/128/50.

We fix the following ACORN index parameters throughout all experiments: maximum degree $M = 16$, navigability parameter $\gamma = 2$, construction beam width $ef_c = 200$, search beam width $ef_s = 200$, and the number of requested neighbors $k = 50$. The stochastic stretch factor used for SBE-Q is held constant at $t = 4.5$ across all runs, as our primary objective is not to tune the expansion radius but rather to (a) validate that the CTR certification-rectification pipeline transfers to filtered search with minimal modification, and (b) study how the predicate selectivity σ modulates the recall-runtime trade-off.

We evaluate three selectivity levels— $\sigma \in \{5, 10, 25\}$ —spanning a representative range. As σ decreases (higher selectivity), ACORN must traverse more of the graph to accumulate k qualifying candidates, increasing the chance of a suboptimal heuristic result and triggering rectification. We set the recall target to $\tau = 0.65$ for all datasets. **Nick: How do we distribute the predicates in the data set to test selectivity? Do we have experiments with a real data set that has predicates?**

For each configuration, we measure two key quantities: (i) the *compliance rate*, defined as the fraction of test queries for which the final output (after any rectification) satisfies $\text{Recall}(q) \geq \tau$, and (ii) the *runtime ratio*, defined as the end-to-end wall-clock time of the CTR pipeline normalized by the standalone ACORN runtime. Normalizing runtime by the ACORN baseline allows direct comparison of the overhead introduced by certification and rectification, independent of the absolute cost of the filtered traversal itself.

Effect of Varying α (Alpha-Sweeps). The parameter α governs the stringency of the statistical certifier. In CRC, α controls the risk level of the conformal procedure: a smaller α requires stronger statistical evidence that the HNSW output meets the recall target before it is accepted without rectification. In LTT, α serves as the significance level of the hypothesis test that the certifier’s error rate does not exceed a prescribed tolerance ϵ .

Figures 14 15 16(a)–(c) present the alpha-sweep results at fixed $\tau = 0.65$ for selectivity levels $\sigma = 5, 10$, and 25 , respectively. Each panel plots the compliance rate (y1 axis) and the normalized runtime (y2 axis) as a function of α , for both CRC and LTT. The baseline ACORN operating point is highlighted for reference.

For CRC, the results reveal a smooth and monotonic trade-off. At small values of α , the certifier is highly conservative: it demands strong statistical evidence before accepting an HNSW result, so a large fraction of queries fail certification and are escalated to the exact SBE-Q + MBV recovery pipeline. This drives the compliance rate close to 1.0—nearly every query ultimately meets the recall target—but at the cost of elevated runtime, since the Dijkstra-based expansion in SBE-Q is substantially more expensive than the heuristic traversal alone. As α increases, the certifier becomes more permissive: it accepts a larger share of HNSW outputs at face value, bypassing rectification. This reduces runtime because fewer queries invoke the expensive recovery procedure, but the compliance rate decreases accordingly, since some of the directly accepted results do not actually meet the target recall.

For LTT exhibits the same qualitative monotonic trade-off but with a notable quantitative difference: the practically useful range of α in LTT is shifted toward substantially smaller values (often by many orders of magnitude). This is an inherent property of the Learn-then-Test mechanism rather than an indication of an unusually stringent confidence requirement. In LTT, the null hypothesis is the boundary case in which exactly x_θ failures occur among m_θ certified queries under a Binomial(m_θ, ϵ) model. The corresponding one-sided tail probability becomes extremely small as soon as the observed failure count drops even slightly below the nominal level $\epsilon \cdot m_\theta$. As a result, the test rejects the null (and therefore certifies the query) only at very small α values, compressing the effective operating range. Despite this shift, the shape of the compliance-versus-runtime frontier mirrors that of CRC: decreasing α pushes

more queries through rectification, raising both compliance and runtime, while increasing α does the reverse.

Effect of Varying τ (Tau-Sweeps). The tau-sweep experiments, presented in Figure 17 18 19, fix the certifier parameters and vary the target recall threshold τ . Each panel reports three quantities as a function of τ : the compliance rate of the CTR pipeline (blue), the compliance rate of standalone ACORN without any certification or rectification (green), and the runtime ratio of the CTR pipeline relative to ACORN (red).

The results reveal a striking complementary pattern. As τ increases (i.e., stricter recall requirements), ACORN’s native compliance rate declines steeply: at high recall thresholds, fewer and fewer of ACORN’s heuristic results happen to contain a sufficiently large overlap with the true k -nearest filtered neighbors. In contrast, the CTR pipeline maintains a substantially higher compliance rate across the full range of τ . This is because the certifier detects the queries for which ACORN’s output is deficient and escalates them to exact recovery via SBE-Q with MBV, which is guaranteed to return the true filtered neighbors. The gap between the green (ACORN) and blue (CTR) curves quantifies the value added by the certify-then-rectify mechanism: it is precisely those queries that would have silently returned low-recall results under ACORN alone that are caught and corrected. The red runtime-ratio curve increases with τ , reflecting the fact that higher recall targets cause more queries to fail certification and require rectification. This increase is gradual at low-to-moderate τ values (where ACORN already achieves reasonable recall for most queries) and steepens at high τ values as the fraction of rectified queries grows.

Effect of Varying the Selectivity Parameter (σ). The alpha-sweep figures report runtime as a *ratio* relative to ACORN’s own runtime rather than as absolute latency. This normalization removes a large shared runtime component (the filtered traversal itself) and consequently compresses between-selectivity differences. In our experiments, ACORN’s absolute runtime is already fairly similar across selectivity levels (especially on SIFT1M), so the dominant visible variation within each panel comes from the α -driven certification and rectification behavior rather than from σ directly.

Nevertheless, the effect of selectivity is discernible, particularly at the high-runtime end of the alpha frontier (i.e., at small α , where rectification is triggered most frequently). At this end, the runtime ratio *decreases* as selectivity decreases from $\sigma = 5$ to $\sigma = 25$. The mechanism is as follows: when the predicate is less selective ($\sigma = 25$), a larger fraction of the database satisfies the filter, so ACORN’s heuristic traversal is more likely to encounter qualifying candidates and return a result that already meets the recall target. Consequently, fewer queries require rectification, and the average per-query overhead shrinks. Conversely, at $\sigma = 5$, the predicate is highly restrictive; ACORN must traverse deeper into the graph to find k qualifying neighbors, increasing the chance of a suboptimal result and triggering rectification more often. Thus, selectivity effects manifest primarily as vertical shifts in the runtime axis rather than as changes in the overall shape of the compliance curve.

Concluding Remarks. Taken together, these results validate that the Certify-then-Rectify framework transfers naturally to the filtered search setting. Across both SIFT1M and DEEP1M and across selectivity levels ranging from highly restrictive ($\sigma = 5$) to moderately selective ($\sigma = 25$), the CTR pipeline consistently achieves

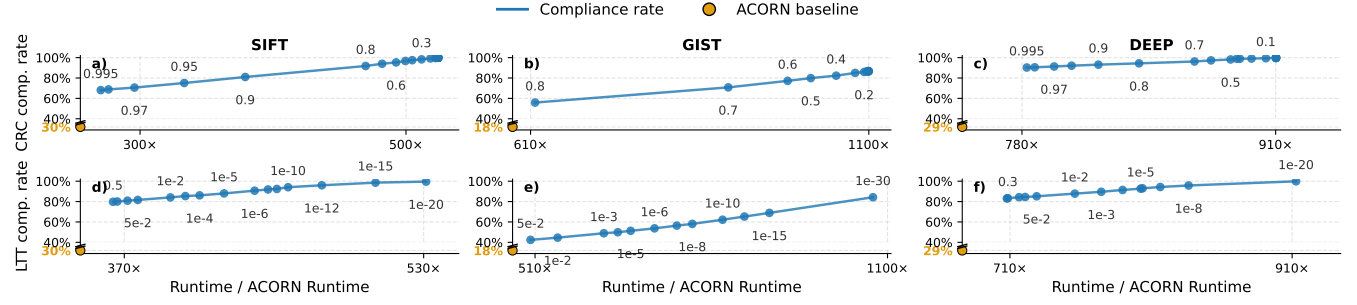


Figure 14: α -sweep on SIFT1M, GIST1M, DEEP1M ($\sigma=5\%$): CRC (top), LTT (bottom). Blue = method frontier vs risk level α (x: runtime/ACORN, log); gold = ACORN baseline.

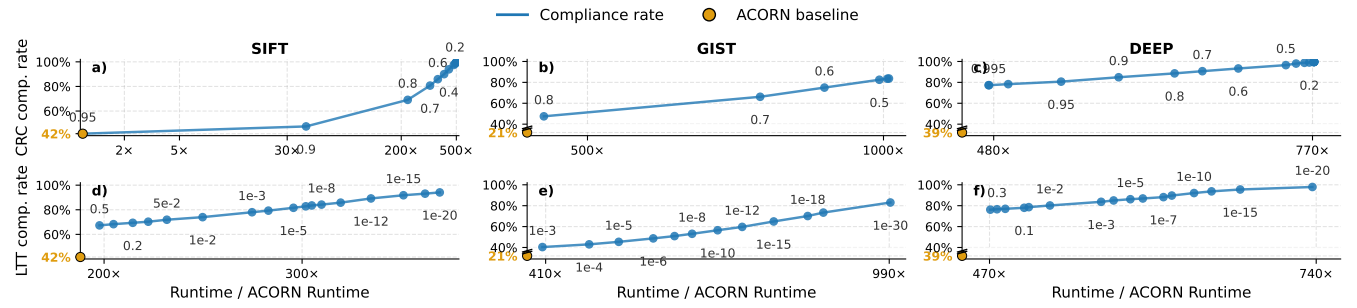


Figure 15: α -sweep on SIFT1M, GIST1M, DEEP1M ($\sigma=10\%$). Panels as in Fig. 14.

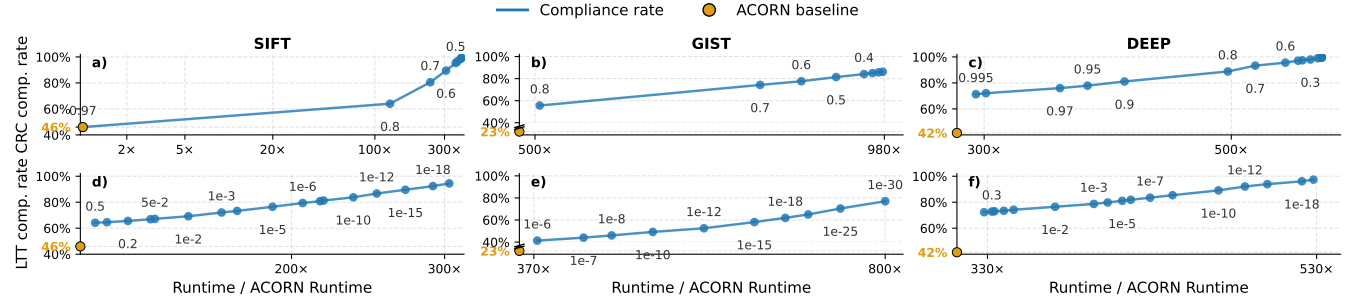


Figure 16: α -sweep on SIFT1M, GIST1M, DEEP1M ($\sigma=25\%$). Panels as in Fig. 14.

substantially higher compliance with the target recall than standalone ACORN, at a controlled and tunable runtime overhead. The α parameter provides a smooth knob for navigating this trade-off: conservative settings (small α) prioritize correctness, while permissive settings (large α) prioritize speed. As part of future efforts, we plan to investigate how information about different predicates—including their distribution, cardinality, and proximity to the query point—can be exploited to further optimize the framework. In particular, we intend to study whether predicate-specific stretch values can reduce the overhead incurred during rectification for highly restrictive filters, and whether adaptive calibration strategies can improve the certifier’s discrimination when the predicate selectivity varies across queries within a single workload.

9 RELATED WORK

A dominant family of graph indexing and ANN search algorithms stems from proximity-construction methods grounded in the geometry of relative neighborhood graphs (RNGs) and approximate Delaunay graphs [42, 46]. Key examples include navigable indexes such as Navigable Small-World graphs (NSW) and their hierarchical HNSW variant [36, 37], which constitute the state-of-the-art. Given its success, many variants of the vanilla HNSW have been proposed in recent literature [33, 40, 41, 56]. Related approaches further include navigating spread-out graphs (NSGs) [22] and the disk-based DiskANN index [27]. Efforts have also been made to accelerate kNN search by compressing the vector space using quantization [29] and inverted file (IVF) indexing [19, 28], and using

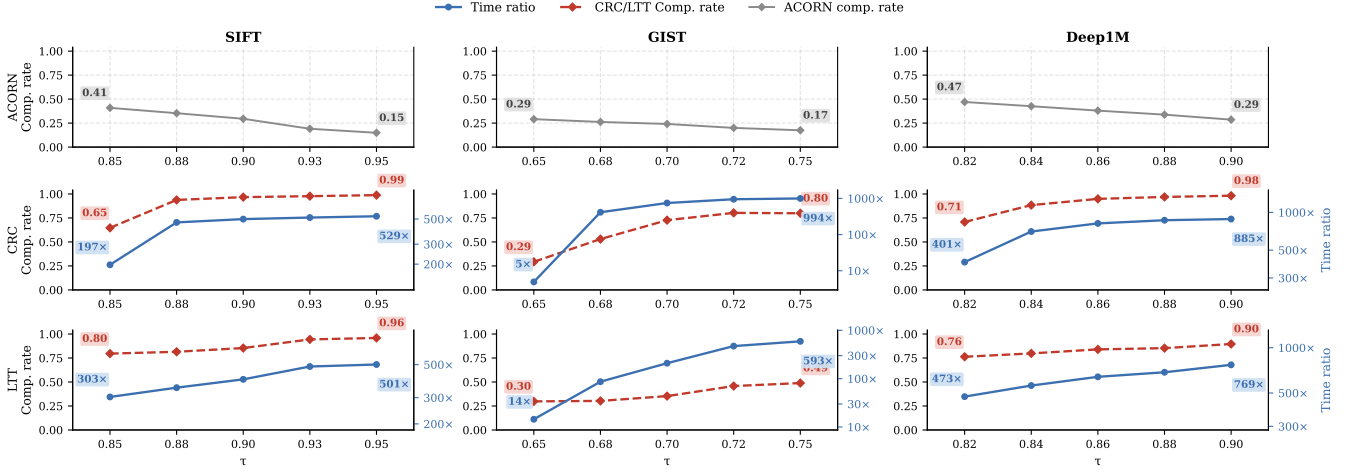


Figure 17: τ -sweep certification on SIFT1M, GIST1M, DEEP1M ($\sigma=5\%$). Top: ACORN natural compliance (gray). Mid/bottom: CRC, LTT compliance (red, left) + runtime/ACORN (blue, right, log) vs τ .

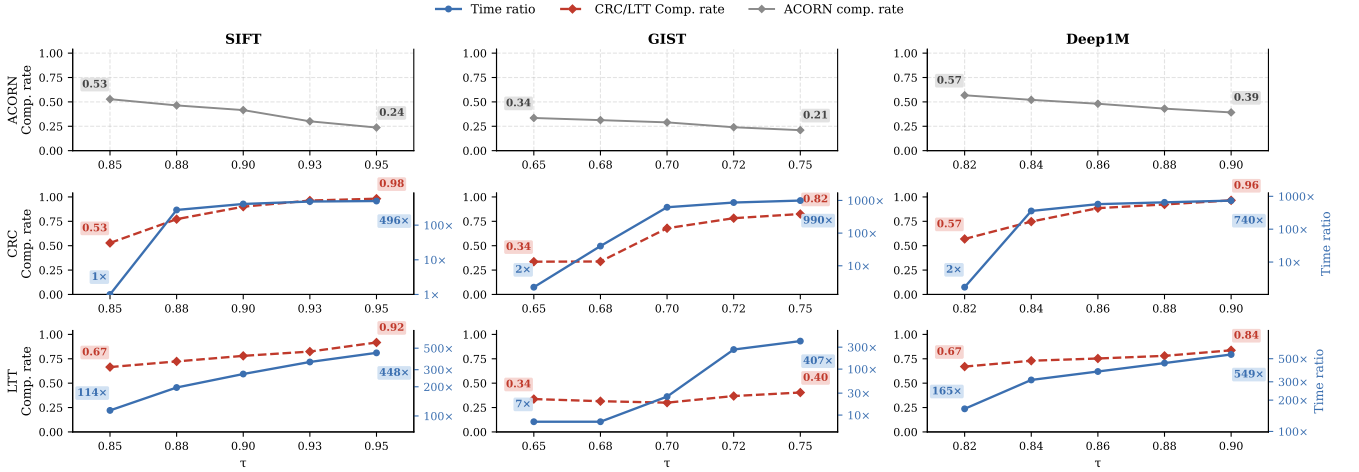


Figure 18: τ -sweep on SIFT1M, GIST1M, DEEP1M ($\sigma=10\%$). Panels as in Fig. 17.

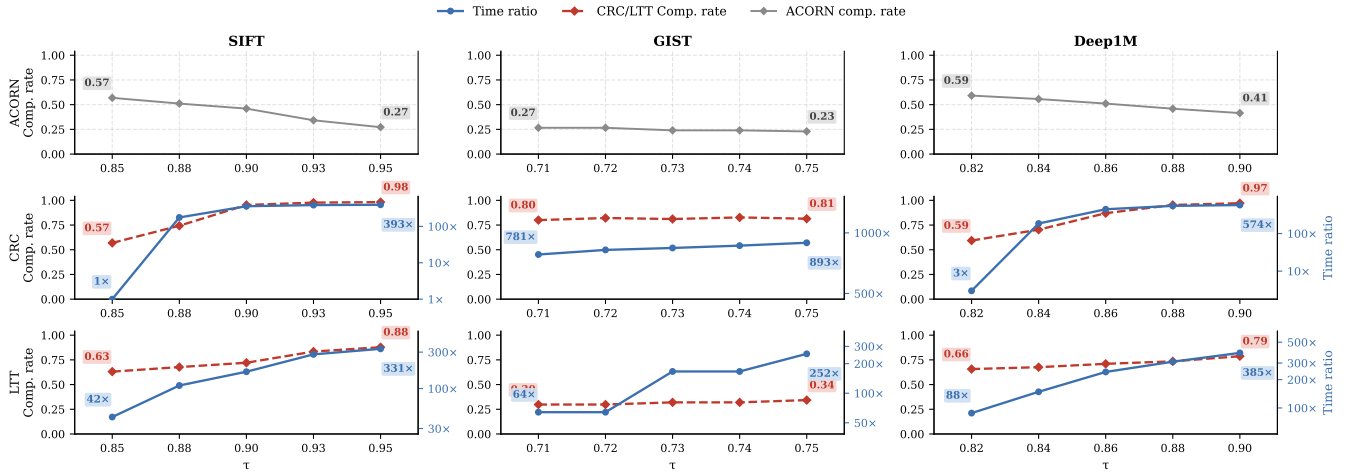
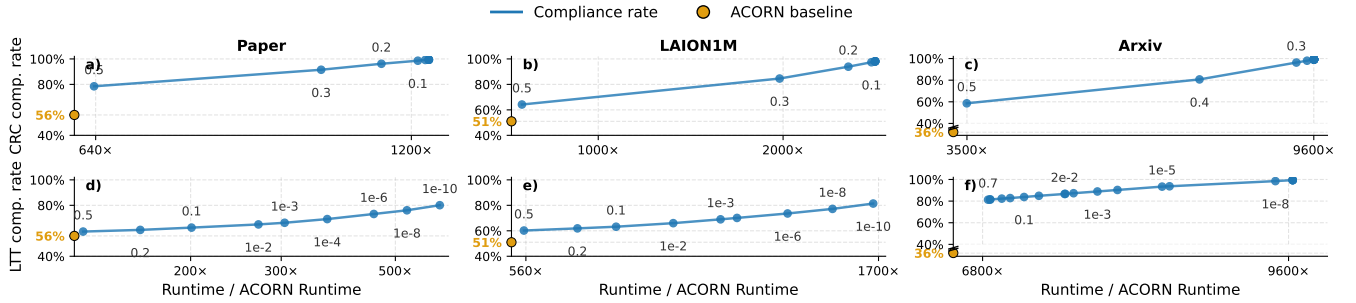
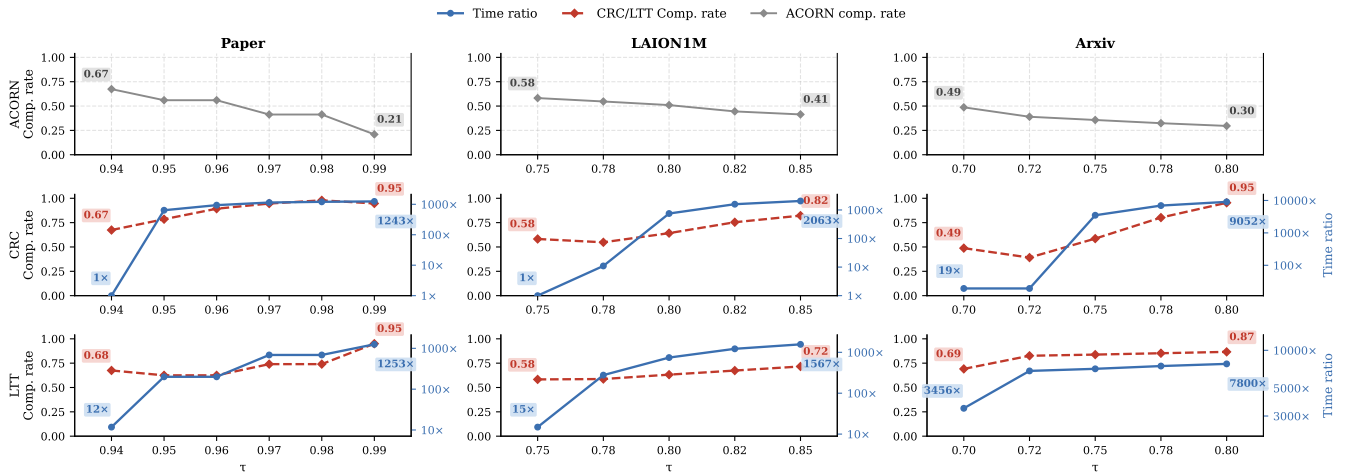
locality-sensitive hashing [11, 17, 35]. A comprehensive survey on ANN search appears in [39]. For filtered search, the literature can be split into: (i) works on filter-aware index structures [9, 33, 50, 56]; (ii) standard ANN backbones with improved query-time execution [1, 40]; and (iii) multi-index approaches [32]. A survey of filtered search methods is given in [15].

Beyond improving the search algorithm itself, recent work has emphasized that query difficulty and workload composition strongly affect the observed behavior of ANN indexes. Wang et al. [47] introduced Steiner-hardness, a query hardness measure specifically designed for graph-based ANN indexes, while Ceccarello et al. [10] studied how to evaluate and generate query workloads of controlled difficulty for high-dimensional vector similarity search. Recent work further explores learned termination checks, where models map query-time features to per-query budgets [30, 51, 53].

Related declarative target-recall approaches terminate when the system *predicts* (heuristically) that the target recall has been met (e.g., DARTH [12]). Similar early-exit ideas also appear in non-graph pipelines [8, 13, 25, 38, 54]. We also build on statistical models grounded in extreme value theory, including standard inferential toolkits for Weibull distributions [16, 21] and conformal prediction [3–5, 7]. Lastly, note that our formulations w.r.t. distance-call pruning take inspiration from recent literature on this topic [6, 18, 23, 30, 31, 43, 44, 49, 52].

10 FINAL REMARKS

This paper introduces a framework for generating correctness certificates in ANN search by wrapping HNSW in a "Certify-then-Rectify" pipeline. Using a Conformal Prediction formulation, we

Figure 19: τ -sweep on SIFT1M, GIST1M, DEEP1M ($\sigma=25\%$). Panels as in Fig. 17.Figure 20: α -sweep on Paper, LAION1M, arXiv: CRC (top), LTT (bottom). Blue = method frontier vs risk level α (x: runtime/ACORN, log); gold = ACORN baseline.Figure 21: τ -sweep certification on Paper, LAION1M, arXiv. Top: ACORN natural compliance (gray). Mid/bottom: CRC, LTT compliance (red, left) + runtime/ACORN (blue, right, log) vs τ .

efficiently certify whether HNSW's top-k results miss closer neighbors. If certification fails, we trigger exact recovery to fetch true top-k neighbors. Our key novelty is reinterpreting HNSW's bottom layer as an empirical graph spanner to estimate a high-confidence stretch factor, bounding the exact recovery search radius. Recovery uses Stretch-Bounded Expansion from Query (SBE-Q) and Metric Bound Verification (MBV) to prune distance computations while preserving correctness. The framework supports filtered search without predicate-respecting paths.

REFERENCES

- [1] Anas Ait Aomar, Karima Echihabi, Marco Arnaboldi, Ioannis Alagiannis, Damien Hilloulin, and Manal Cherkaoui. 2025. RWalks: Random Walks as Attribute Diffusers for Filtered Vector Search. *PACMMOD* 3, 3 (2025).
- [2] Yousef Al-Jazzazi, Haya Diwan, Jinrui Gou, Cameron Musco, Christopher Musco, and Torsten Suel. 2025. Distance Adaptive Beam Search for Provably Accurate Graph-Based Nearest Neighbor Search. *arXiv preprint arXiv:2505.15636* (2025).
- [3] Anastasios N Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511* (2021).
- [4] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, and Michael I. Jordan. 2024. Conformal Risk Control. *J. Amer. Statist. Assoc.* 119, 548 (2024), 2386–2408. <https://doi.org/10.1080/01621459.2024.2302500>
- [5] Anastasios N. Angelopoulos, Stephen Bates, Michael I. Jordan, and Jitendra Malik. 2022. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. In *ICLR*. OpenReview.
- [6] Jeess Augustine, Suraj Shetiya, Mohammadreza Esfandiari, Senjuti Basu Roy, and Gautam Das. 2021. A Generalized Approach for Reducing Expensive Distance Calls for A Broad Class of Proximity Problems. In *ACM SIGMOD*. 142–154.
- [7] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. 2021. Distribution-free, Risk-controlling Prediction Sets. *J. ACM* 68, 6, Article 43 (Sept. 2021), 34 pages. <https://doi.org/10.1145/3478535>
- [8] Francesco Busolin, Claudio Lucchese, Franco Maria Nardini, Salvatore Orlando, Raffaele Perego, and Salvatore Trani. 2024. Early exit strategies for approximate k-NN search in dense retrieval. In *CIKM*. 3647–3652.
- [9] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *PACMMOD* 2, 6 (2024).
- [10] Matteo Ceccarello, Alexandra Levchenko, Ioana Ileana, and Themis Palpanas. 2025. Evaluating and Generating Query Workloads for High Dimensional Vector Similarity Search. In *ACM SIGKDD*.
- [11] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. 380–388.
- [12] Manos Chatzakos, Yannis Papakonstantinou, and Themis Palpanas. 2025. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *PACMMOD* 3, 4 (2025), 1–26.
- [13] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, Vol. 34.
- [14] Yilun Chen, Zhiding Yu, Yukang Chen, Shiyi Lan, Anima Anandkumar, Jiaya Jia, and Jose M. Alvarez. 2023. FocalFormer3D: Focusing on Hard Instance for 3D Object Detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- [15] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *PVLDB* 18, 12 (2025), 5488–5492.
- [16] Stuart Coles, Joanna Bawa, Lesley Trenner, and Pat Dorazio. 2001. *An introduction to statistical modeling of extreme values*. Vol. 208. Springer.
- [17] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SCG*. 253–262.
- [18] Liwei Deng, Penghao Chen, Ximu Zeng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. Efficient Data-Aware Distance Comparison Operations for High-Dimensional Approximate Nearest Neighbor Search. *PVLDB* 18, 3 (2024), 812–821.
- [19] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvassy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). [arXiv:2401.08281](https://arxiv.org/abs/2401.08281) [cs.LG]
- [20] Hao Duan, Yitong Song, Bin Yao, and Anqi Liang. 2025. PGTuner: An Efficient Framework for Automatic and Transferable Configuration Tuning of Proximity Graphs. [arXiv:2508.17886](https://arxiv.org/abs/2508.17886) [cs.DB] <https://arxiv.org/abs/2508.17886>
- [21] R. A. Fisher and L. H. C. Tippett. 1928. Limiting forms of the frequency distribution of the largest or smallest member of a sample. *Mathematical Proceedings of the Cambridge Philosophical Society* 24, 2 (1928), 180–190.
- [22] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB* 12, 5 (2019), 461–474.
- [23] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *PACMMOD* 1, 2 (2023).
- [24] E. J. Gumbel. 1958. *Statistics of Extremes*. Columbia University Press, New York, Chichester, West Sussex. <https://doi.org/10.7312/gumb92958>
- [25] Sonia Horchidan, Fabian Zeiher, Henrik Boström, and Paris Carbone. 2026. Co-ANN: Conformal Approximate Nearest Neighbor Search. *PVLDB* 19, 1 (2026), 29–42.
- [26] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haochuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. 2025. $\pi_{0,6}^*$: a VLA That Learns From Experience. [arXiv:2511.14759](https://arxiv.org/abs/2511.14759) [cs.LG] <https://arxiv.org/abs/2511.14759>
- [27] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*, Vol. 32.
- [28] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [29] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [30] Conglong Li, Minjia Zhang, David G Andersen, and Yuxiong He. 2020. Improving approximate nearest neighbor search through learned adaptive early termination. In *ACM SIGMOD*. 2539–2554.
- [31] Zhenxin Li, Shuibing He, Jiahao Guo, Xuechen Zhang, Xian-He Sun, and Gang Chen. 2025. CROUTING: Reducing Expensive Distance Calls in Graph-Based Approximate Nearest Neighbor Search. [arXiv preprint arXiv:2509.00365](https://arxiv.org/abs/2509.00365) (2025).
- [32] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *PVLDB* 18, 11 (2025), 4723–4736.
- [33] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *PVLDB* 18, 4 (2024), 1118–1130.
- [34] Zhenxi Lin, Ziheng Zhang, Xian Wu, and Yefeng Zheng. 2023. Improving Biomedical Entity Linking with Retrieval-Enhanced Learning. [arXiv preprint arXiv:2312.09806](https://arxiv.org/abs/2312.09806) (2023). <https://doi.org/10.48550/arXiv.2312.09806>
- [35] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *Vldb*. 950–961.
- [36] Yuri Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [37] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
- [38] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association.
- [39] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *Vldb* 33, 5 (2024), 1591–1615.
- [40] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *PACMMOD* 2, 3 (2024), 1–27.
- [41] Zhencan Peng, Miao Qiao, Wenqiao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *PVLDB* 18, 10 (2025).
- [42] Franco P Preparata and Michael I Shamos. 1982. *Computational Geometry: An Introduction*. Springer Science & Business Media.
- [43] Yitong Song, Pengcheng Zhang, Chao Gao, Bin Yao, Kai Wang, Zongyuan Wu, and Lin Qu. 2025. TRIM: Accelerating High-Dimensional Vector Similarity Search with Enhanced Triangle-Inequality-Based Pruning. *PACMMOD* 3, 6 (2025), 1–26.
- [44] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations.

- 2437 *PACMOD* 3, 6 (2025), 1–29.
- 2438 [45] Tommaso Teofili and Jimmy Lin. 2025. Patience In Proximity: A Simple Early
2439 Termination Strategy for HNSW Graph Traversal in Approximate k-Nearest
2440 Neighbor Search. In *ECIR*. 401–407.
- 2441 [46] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar
2442 set. *Pattern Recognition* 12, 4 (1980), 261–268.
- 2443 [47] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and
2444 Wei Wang. 2024. Steiner-Hardness: A Query Hardness Measure for Graph-Based
2445 ANN Indexes. *PVLDB* 17, 13 (2024), 4668–4682.
- 2446 [48] Maciej Wiatrak, Eirini Arvaniti, Angus Brayne, Jonas Vetterle, and Aaron Sim.
2023. Proxy-based Zero-Shot Entity Linking by Effective Candidate Retrieval.
2447 *arXiv preprint arXiv:2301.13318* (2023). <https://doi.org/10.48550/arXiv.2301.13318>
- 2448 [49] Qian Xu, Juan Yang, Feng Zhang, Junda Pan, Kang Chen, Youren Shen, Amelie Chi
2449 Zhou, and Xiaoyong Du. 2025. Tribase: A Vector Data Query Engine for Reliable
2450 and Lossless Pruning Compression using Triangle Inequalities. *PACMOD* 3, 1
2451 (2025), 1–28.
- 2452 [50] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen.
2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering
2453 Nearest Neighbor Search. *PACMOD* 2, 6 (2024).
- 2454 [51] Kaixiang Yang, Hongya Wang, Bo Xu, Wei Wang, Yingyuan Xiao, Ming Du, and
2455 Junfeng Zhou. 2021. Tao: A learning framework for adaptive nearest neighbor
2456 search using static features only. *arXiv preprint arXiv:2110.00696* (2021).
- 2457 [52] Mingyu Yang, Wentao Li, Jiabao Jin, Xiaoyao Zhong, Xiangyu Wang, Zhitao
2458 Shen, Wei Jia, and Wei Wang. 2025. Effective and general distance computation
2459 for approximate nearest neighbor search. In *IEEE ICDE*. 1098–1110.
- 2460 [53] Chao Zhang and Renée J. Miller. 2025. Distribution-Aware Exploration for
2461 Adaptive HNSW Search. *arXiv:2512.06636 [cs.DB]* <https://arxiv.org/abs/2512.06636>
- 2462 [54] Zili Zhang, Chao Jin, Linpeng Tang, Xuanzhe Liu, and Xin Jin. 2023. Fast, Ap-
2463 proximate Vector Queries on Very Large Unstructured Datasets. In *20th USENIX
Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX
2464 Association, 995–1011.
- 2465 [55] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu
2466 Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen,
2467 Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework
2468 for Graph-Based Approximate Nearest Neighbor Search. *PVLDB* 18, 12 (2025),
2469 5017–5030.
- 2470 [56] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Seg-
2471 ment Graph for Range-Filtering Approximate Nearest Neighbor Search. *PACM-
MOD* 2, 1 (2024).